
Improving the Performance and Learning Stability of Parallelizable RNNs Designed for Ultra-Low Power Applications

Julien Brandoit¹ Arthur Fyon¹ Damien Ernst¹ Guillaume Drion¹

Abstract

Sequence learning is dominated by Transformers and parallelizable recurrent neural networks (RNNs) such as state-space models, yet learning long-term dependencies remains challenging, and state-of-the-art designs trade power consumption for performance. The Bistable Memory Recurrent Unit (BMRU) was introduced to enable hardware-software co-design of ultra-low power RNNs: quantized states with hysteresis provide persistent memory while mapping directly to analog primitives. However, BMRU performance lags behind parallelizable RNNs on complex sequential tasks. In this paper, we identify gradient blocking during state updates as a key limitation and propose a cumulative update formulation that restores gradient flow while preserving persistent memory, creating skip-connections through time. This leads to the Cumulative Memory Recurrent Unit (CMRU) and its relaxed variant, the α CMRU. Experiments show that the cumulative formulation dramatically improves convergence stability and reduces initialization sensitivity. The CMRU and α CMRU match or outperform Linear Recurrent Units (LRUs) and minimal Gated Recurrent Units (minGRUs) across diverse benchmarks at small model sizes, with particular advantages on tasks requiring discrete long-range retention, while the CMRU retains quantized states, persistent memory, and noise-resilient dynamics essential for analog implementation.

¹Department of Electrical Engineering and Computer Science, University of Liège, Liège, Belgium. Correspondence to: Julien Brandoit <jbrandoit@uliege.be>, Guillaume Drion <gdrion@uliege.be>.

1. Introduction

The success of deep learning on sequential data has been driven by increasingly large models with increasingly large computational costs. Transformers achieve state-of-the-art performance through attention mechanisms, but their quadratic complexity in sequence length makes them impractical for resource-constrained deployment. Recurrent neural networks (RNNs) offer a more efficient alternative, encoding temporal dependencies in a fixed-size hidden state. However, classical RNNs rely on nonlinear recurrence, creating sequential dependencies that prevent parallel training.

A growing body of work has propelled parallelizable RNNs by adopting linear state dynamics as a deliberate architectural constraint. This design choice precludes nonlinear recurrence but is the structural property these models leverage to parallelize computation via associative scan algorithms. State-space models (SSMs) and parallelizable gated RNNs exploit this linearity to train increasingly large architectures, matching or exceeding Transformer performance on many benchmarks.

Beyond this notable boost in performance, modern parallelizable RNNs face two fundamental challenges. The first concerns hardware efficiency. Current RNN implementations, whether linear or nonlinear, are emulated on digital architectures: CPUs, GPUs, microcontrollers, or FPGAs (Fedorov et al., 2019; Chang et al., 2015). Even optimized TinyML deployments consume milliwatts of power (Banbury et al., 2021; Warden & Situnayake, 2019). Achieving true sub-microwatt operation, as required for *always-on* sensors or biomedical implants with decade-long battery life, demands RNNs with computational primitives that map directly onto efficient analog circuits rather than being emulated in digital logic. The second concerns memory. The contractive dynamics adopted by parallelizable RNNs, with state eigenvalues kept inside the unit circle to ensure stability, force states to decay exponentially toward a unique equilibrium, a property known as *fading memory*. While fading memory suffices for many tasks, it precludes *persistent memory*, which is the ability to retain information indefinitely until explicitly overwritten. Persistent memory can only be emulated by such architectures through increased network size, which is to be avoided in low-power

applications. *True, efficient persistent memory requires multistability, which contractive dynamics cannot exhibit.*

Bistable Memory Recurrent Units (BMRUs) have been specifically designed to address both challenges. BMRUs enable persistent memory through bistability between two *quantized* hidden states, while remaining parallelizable via associative scans. This bistable, quantized structure maps directly to Schmitt triggers, a well-known analog primitive. It enables ultra-low power implementation in monolithic CMOS technology: a recent design achieves $>90\%$ accuracy at 100 nW for keyword spotting, three orders of magnitude below digital alternatives (Fyon et al., 2026b;a). Quantized states also block analog noise propagation, preventing error accumulation across timesteps. However, persistent memory comes at the cost of expressivity. BMRUs lack the fading memory that SSMs and gated RNNs exploit for complex sequential computations, limiting performance on tasks requiring more than long-term retention. This raises a natural question: *can BMRUs maintain their persistent memory and hardware co-design properties while approaching the performance of SSMs and gated parallelizable RNNs on more demanding sequential tasks?*

In this paper, we tackle this question by first identifying a key limitation of BMRUs: the full state replacement in update mode blocks gradient flow during training. We introduce a cumulative update formulation that restores gradient flow, dramatically improving convergence and reducing initialization sensitivity while preserving persistent memory. Building on this formulation, we propose the Cumulative Memory Recurrent Unit (CMRU) and the α CMRU. The CMRU retains quantized states and maps directly to analog primitives, now corresponding to a hysteretic charge accumulator, making it suitable for ultra-low power analog implementations. The α CMRU relaxes quantization constraints, providing increased expressivity for large-scale digital machine learning. We evaluate our approach with a focus on small model sizes and interpretability, motivated by the target application of ultra-low power analog electronics. Our experiments demonstrate that the CMRU and α CMRU bridge the performance gap with SSMs and parallelizable gated RNNs while addressing distinct deployment scenarios: analog hardware for the CMRU and classical digital systems for the α CMRU.

Our contributions can be summarized as follows:

1. We identify gradient blocking during BMRU state updates as the key training bottleneck and show that the cumulative update formulation resolves it, making parallelizable persistent memory a reliable and competitive machine learning primitive for the first time.
2. We introduce the Cumulative Memory Recurrent Unit (CMRU) and its relaxed variant, the α CMRU, as novel

parallelizable RNN cells providing persistent memory. The CMRU is structurally compatible with associative-scan pipelines and can serve as a drop-in persistent-memory block in Mamba-style architectures. The (α)CMRU admits a selective state-space representation in which the hyperparameter ε directly controls the eigenvalues of the state transition matrix during updates, unifying the cumulative and bistable regimes under a single parameterization and connecting persistent memory to the broader theory of linear recurrent models.

3. Through systematic benchmarking, we establish a complementary performance pattern: CMRU variants outperform fading-memory cells on tasks requiring discrete state preservation, while fading-memory cells retain an advantage on tasks requiring smooth temporal integration. This specialization motivates hybrid architectures combining both memory types.
4. We demonstrate practical ultra-low power deployment: a single-layer CMRU with hidden state dimension $d \leq 16$ achieves $>95\%$ accuracy on keyword spotting benchmarks. The ε -annealing training strategy produces models that deploy directly on existing validated ultra-low power hardware (Fyon et al., 2026a) at sub-microwatt power levels. We additionally provide a proof-of-concept native CMRU analog circuit, validated in subthreshold CMOS simulation.

2. Background

RNNs encode time-dependencies in a recurrent hidden state h_t whose update depends on the previous hidden state h_{t-1} and current observation of a time-series x_t . It writes

$$h_t = f_\theta(x_t, h_{t-1}; \theta), \forall t \geq 1, \quad (1)$$

where f_θ represents the state transition equation of the RNN and θ the learnable parameters of the network. This recurrent connection enables information persistence (or *memory*) across time through encoding in its hidden state. Nonlinear versions such as LSTM (Hochreiter & Schmidhuber, 1997) and GRUs (Cho et al., 2014) have dominated sequence modeling for decades. However, their sequential dependencies, arising from nonlinear gating mechanisms, prevent parallel training and energy-efficient analog implementation (Martin & Cundy, 2018; Heydari & Mahmoud, 2025).

2.1. Related Work on Parallelizable RNNs

Recent efforts have led to the emergence of novel RNN architectures that are parallelizable over the sequence length with competitive performance on sequential benchmarks.

State-space models (SSMs) and linear RNNs exploit linear state dynamics (the f_θ is affine in h_{t-1}) to achieve paralleliz-

able training, either through convolutional representations (Gu et al., 2021; 2022b) or the parallel scan algorithm (Blelloch, 1989; Martin & Cundy, 2018; Smith et al., 2023; Gu & Dao, 2024). The Linear Recurrent Unit (LRU) (Orvieto et al., 2023) distills this family to its essential form: a diagonal linear recurrence with complex-valued states and exponential parameterization for stability. We adopt LRU as our representative of this family of architectures in our experiments for its simplicity and competitive performance. This cell is presented in detail in Section F.7.1.

Gated RNNs can also achieve parallelizable training by removing the state-dependency in their gating mechanisms, i.e. computing gate values from the input alone, $z_t = \sigma(W_z x_t)$. It yields an update of the form $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$, where $\tilde{h}_t$ depends only on x_t . As in SSMs, this is a linear recurrence in h_t (the f_θ is affine in h_{t-1}) and can thus be parallelized via the scan algorithm. Several such architectures have been proposed, including xLSTM (Beck et al., 2024), HGRN (Qin et al., 2023), MatMul-free LM (Zhu et al., 2024) and GILR (Martin & Cundy, 2018), though some only achieve partial parallelization. We adopt min-GRU (Feng et al., 2024) as our representative of this family of architectures in our experiments: it strips the GRU to its minimal form (a single input-dependent gate and no reset gate) making it one of the simplest fully parallelizable gated architectures. This cell is presented in detail in Section F.7.2.

In all these models, their structure implies monostability, representative of *fading memory* (Boyd & Chua, 1985), as they force the eigenvalues of the discrete update laws to strictly lie within the unit circle. A broader discussion of related work, including hardware-oriented and hybrid architectures, is provided in Section A.

2.2. The Bistable Memory Recurrent Unit

The Bistable Memory Recurrent Unit (BMRU) has been recently introduced as an alternative RNN cell that achieves persistent memory while preserving parallelizability (Geeter et al., 2026). The BMRU hidden state is *quantized*: each component of the hidden state vector h_t can only take one of two values, $+\alpha$ or $-\alpha$. This binary quantization is the signature of bistability. The BMRU dynamics are given by:

$$\hat{h}_t = W_x x_t + b_x, \quad (2)$$

$$\beta_t = |W_\beta x_t + b_\beta|, \quad (3)$$

$$z_t = H(|\hat{h}_t| - \beta_t), \quad (4)$$

$$h_t = z_t \odot S(\hat{h}_t) \odot \alpha + (1 - z_t) \odot h_{t-1}, \quad (5)$$

where $H(\cdot)$ denotes the Heaviside step function, $S(\cdot)$ is the sign function, W_x and W_β are learnable weight matrices, and b_x , b_β , and α are learnable parameter vectors. This formulation implements adaptive thresholding: when the candi-

date magnitude $|\hat{h}_t|$ exceeds the input-dependent threshold β_t , the hidden state switches to $\alpha S(\hat{h}_t)$; otherwise, it remains unchanged. The quantized, bistable nature of the state enables persistent memory: information is retained indefinitely until an input actively triggers a switch.

The interest of BMRUs lies in the direct mapping onto the Schmitt trigger, an analog primitive. This enables ultra-low power analog implementations in monolithic CMOS technology (Fyon et al., 2026a;b). In contrast, fading-memory architectures such as LRUs are typically emulated in digital hardware for low-power applications (microcontrollers, FPGAs) (Banbury et al., 2021; Warden & Situnayake, 2019; Fedorov et al., 2019; Chang et al., 2015); a true analog implementation would require capacitors to realize the exponential decay, precluding monolithic integration.

3. Extending the BMRU: Continuous States and Cumulative Updates

The original BMRU achieves persistent memory through quantized bistable states, but its design introduces two potential limitations: (i) the restriction to binary values $\pm\alpha$ may limit representational capacity, and (ii) the state reset during updates blocks gradient flow, hindering learning. In this section, we introduce the Cumulative Memory Recurrent Unit (CMRU) and its variant the α CMRU. The CMRU addresses the gradient flow limitation through a cumulative update formulation while retaining quantized states for analog implementation. The α CMRU additionally relaxes quantization constraints for large-scale digital machine learning.

3.1. Restoring Gradient Flow: Cumulative Updates

The BMRU suffers from a gradient blocking problem during state updates. The cell operates in two modes depending on the gate z_t . In *retain mode* ($z_t = 0$, when $|\hat{h}_t| < \beta_t$), the hidden state is copied forward unchanged, enabling perfect gradient propagation (see Equation (5)). However, in *update mode* ($z_t = 1$, when $|\hat{h}_t| \geq \beta_t$), the previous state is erased and overwritten, completely blocking gradient flow:

$$\left. \frac{\partial h_t}{\partial h_{t-1}} \right|_{z_t=1} = 0. \quad (6)$$

This prevents error signals from reaching earlier timesteps through update operations, making it difficult for the model to learn effective update strategies via backpropagation through time. The problem is most severe during early training, before the gating mechanism has learned when to retain versus update, resulting in poor convergence and high initialization sensitivity.

Cumulative update formulation. We address gradient blocking by enabling partial state preservation during up-

dates. Introducing a hyperparameter $\varepsilon \in [0, 1]$ that controls the fraction of previous state retained, we define the Cumulative Memory Recurrent Unit (CMRU) through the modified update equation:

$$h_t = z_t \odot \left[S(\hat{h}_t) \odot \alpha + \varepsilon h_{t-1} \right] + (1 - z_t) \odot h_{t-1}. \quad (7)$$

The term εh_{t-1} is active only during updates ($z_t = 1$), creating a temporal skip connection that restores gradient flow (from Equation (7)):

$$\left. \frac{\partial h_t}{\partial h_{t-1}} \right|_{z_t=1} = \varepsilon. \quad (8)$$

The hyperparameter ε interpolates between three regimes.

When $\varepsilon = 0$, the formulation reduces to the original BMRU with full state replacement.

When $0 < \varepsilon < 1$, the model exhibits *non-instantaneous fading memory*. Unlike LRU or minGRU, where the state decays at every timestep regardless of input, here fading occurs only during update events. Consider a sequence of k successive updates at times $t_1, t_2, \dots, t_k$: the contribution of the state at t_1 to h_{t_k} is scaled by ε^{k-1} . Memory thus fades exponentially with the *number of updates*, not with elapsed time. Between updates, the state is preserved perfectly. This creates an event-driven forgetting mechanism: salient inputs (those triggering updates) gradually overwrite older information, while periods of low activity preserve memory indefinitely.

When $\varepsilon = 1$, the cell becomes a *selective integrator*:

$$h_t|_{\varepsilon=1} = h_{t-1} + z_t \odot S(\hat{h}_t) \odot \alpha, \quad (9)$$

accumulating quantized inputs into the hidden state with perfect gradient flow ($\frac{\partial h_t}{\partial h_{t-1}} = 1$).

Critically, the persistent memory property in retain mode ($z_t = 0$) is unchanged for all values of ε .

3.2. Relaxing Quantization: the α CMRU Variant

To investigate whether the CMRU fixed quantization scale limits representational capacity, we introduce the α CMRU, which replaces the fixed scale α with an input-dependent function:

$$\alpha_t = W_\alpha x_t + b_\alpha, \quad (10)$$

where W_α and b_α are learnable parameters. The state update equation becomes:

$$h_t = z_t \odot \left[S(\hat{h}_t) \odot \alpha_t + \varepsilon h_{t-1} \right] + (1 - z_t) \odot h_{t-1}. \quad (11)$$

Unlike the CMRU, whose hidden states are quantized, the α CMRU can represent arbitrary values in $\mathbb{R}^d$.

The two variants address different deployment scenarios with complementary trade-offs. For ultra-low power analog implementations, the CMRU quantized states offer inherent noise resilience. For classical machine learning on digital hardware, the α CMRU continuous representation removes any potential precision constraints, providing increased expressivity for large-scale applications.

3.3. Fixed Point Structure and Representational Capacity

The cumulative formulation fundamentally changes which states the cell can represent.

In the original BMRU ($\varepsilon = 0$), each update overwrites the state with $\pm\alpha$. The cell can therefore only occupy 2^d distinct states, where d is the hidden dimension (one binary choice per dimension). This is the quantized bistable structure.

With $\varepsilon = 1$, updates no longer overwrite but *accumulate*: each update adds $\pm\alpha$ to the current state. The reachable states thus form a regular lattice at integer multiples of α , as illustrated in Figure 1. This expands representational capacity from 2^d discrete states to countably infinite attractors, while maintaining a fixed precision: successive states differ by at least $\|\alpha\|$.

For the α CMRU with $\varepsilon = 1$, each update accumulates a different input-dependent value α_t . Since the α_t vary continuously with the input, the set of reachable states becomes dense in $\mathbb{R}^d$: the cell can in principle approximate any target state with arbitrary precision.

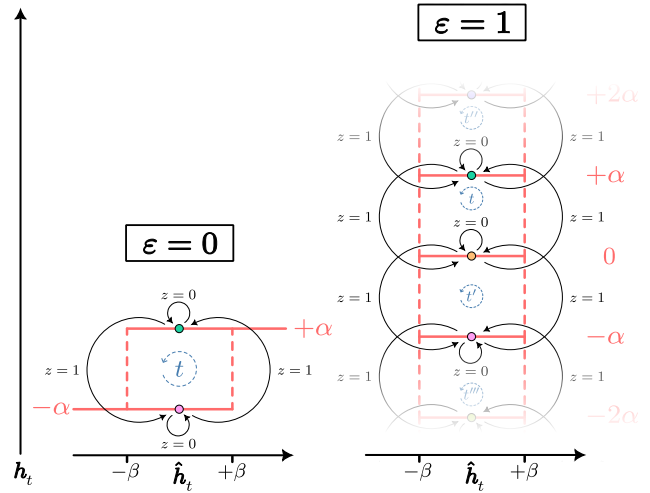


Figure 1. Fixed point structure of hidden state h_t for CMRU as a function of ε . **Left** ($\varepsilon = 0$): Original BMRU with two discrete stable fixed points at $h^* = \pm\alpha$. **Right** ($\varepsilon = 1$): CMRU cumulative formulation expands reachable fixed points to all integer multiples $h^* \in \{\pm k\alpha : k \in \mathbb{N}\}$, creating countably infinite stable attractors. Dashed vertical lines at $\pm\beta$ indicate candidate state thresholds for gating.

3.4. Experimental Framework

Comparing RNN architectures solely on peak benchmark accuracy can be misleading: an architecture that achieves state-of-the-art results under careful tuning may prove brittle in practice if its performance is sensitive to weight initialization or requires extensive hyperparameter search for each new task (Henderson et al., 2018; Bouthillier et al., 2021). To provide a more complete picture, we characterize each architecture through three complementary properties:

- (i) **Peak performance:** the best accuracy achievable by the architecture, reflecting its expressive capacity;
- (ii) **Initialization sensitivity:** the variability in performance across different random seeds, indicating training reliability;
- (iii) **Hyperparameter robustness:** the ability to perform well across diverse tasks without task-specific tuning, facilitating practical and accessible use across different machine learning domains.

We detail our experimental strategy below.

Common backbone architecture. To isolate the effect of the recurrent cell, all cells (LRU, minGRU, CMRU, and α CMRU) are integrated into an identical backbone architecture. The backbone interleaves recurrent layers with point-wise multi-layer perceptrons (MLPs), skip connections, and normalization layers, as illustrated in Figure 5. Full architecture and training details are provided in Section F.

No hyperparameter tuning. Critically, we perform no hyperparameter tuning across tasks. A single set of hyperparameters is used for all experiments. The validation set serves exclusively for early stopping and checkpoint selection; it is never used to inform hyperparameter decisions. This protocol ensures that our results reflect the inherent robustness of each architecture rather than the outcome of task-specific optimization, and allows us to assess property (iii) implicitly through consistent performance across diverse tasks.

Evaluation protocol. Properties (i) and (ii) are assessed by training each configuration with five independent random seeds and evaluating the best checkpoint on the test set. We report accuracy for classification benchmarks and mean absolute error (MAE) for regression benchmarks. Results are summarized as mean $\pm$ min-max intervals, capturing both peak performance and initialization sensitivity. Numerical values are tabulated in Section I, and raw training logs are publicly available alongside the code.¹

¹<https://github.com/julienbrandoit/ICML2026---Improving-Performance-and-Stability-of-Ultra-Low-Power-RNNs.git>

4. Experimental Investigation of the (α)CMRU

4.1. Effect of ε on Gradient Flow and Convergence

Throughout this section, we evaluate models using *last pooling*. This choice tests both retention of information over the entire sequence and the ability to learn from long-range dependencies. Detailed descriptions of all benchmark tasks used in this work are provided in Section G.

We first investigate the impact of varying ε on the sequential MNIST (sMNIST) classification task, where models classify handwritten digits presented pixel-by-pixel in raster scan order. Increasing ε from 0 to 1 yields consistent improvements in mean accuracy and initialization stability across both CMRU and α CMRU variants (Figure 2, left panel). For CMRU with $d = 32$, accuracy improves from approximately 30% at $\varepsilon = 0$ (i.e., the original BMRU) to 96% at $\varepsilon = 1$, while cross-seed variability decreases dramatically. The α CMRU exhibits similar trends, achieving 97% accuracy at $\varepsilon = 1$ with minimal variance. These improvements occur despite different quantization mechanisms (fixed versus input-dependent), confirming that gradient flow through cumulative updates (not representational capacity) is the critical factor enabling reliable convergence. Peak performance occurs at $\varepsilon = 1$, where gradient magnitude equals unity in both retain and update modes. This observation remains consistent across different permutations of the sequential MNIST task (see Figure 6 in Section H.1).

4.2. Scaling Behavior and Initialization Robustness

Beyond addressing gradient flow, we investigate whether cumulative updates improve robustness across different model capacities. We compare performance on sMNIST across different state dimensions d (Figure 2, right panel).

Both CMRU and α CMRU with $\varepsilon = 1$ achieve higher accuracy and lower cross-seed variability than LRU and minGRU across all tested dimensions. The advantage is most pronounced at small state sizes, which is critical for ultra-low power implementations: at $d = 4$, CMRU achieves approximately 84% accuracy with minimal variance, while LRU and minGRU exhibit substantially higher variability and lower mean performance. At $d = 256$, all architectures converge to similar accuracy (96–97%), but the CMRU variants maintain lower variance.

This reduced initialization sensitivity across all scales confirms that cumulative updates yield more stable optimization landscapes, an essential property for hardware-constrained deployments and large-scale machine learning where training costs are prohibitive. The consistent behavior across dimensions suggests that these improvements are fundamental to the architecture rather than artifacts of a particular capacity regime.

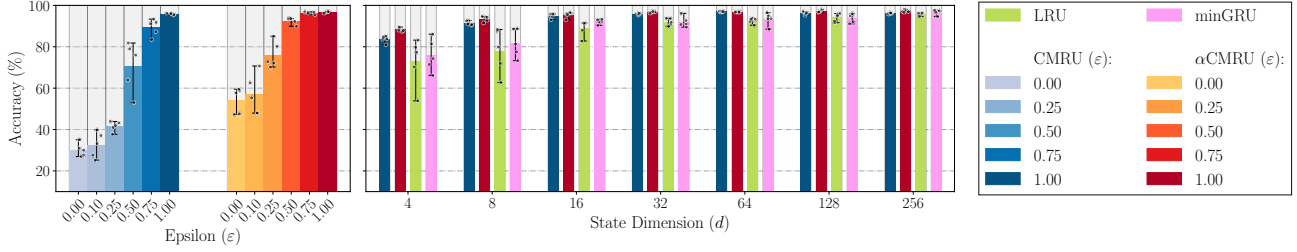


Figure 2. Performance on sequential MNIST (sMNIST) classification task. Results show mean accuracy across five random initializations with min-max error bars. **Left:** Effect of hyperparameter $\epsilon \in \{0.00, 0.25, 0.50, 0.75, 1.00\}$ on CMRU and α CMRU ($d = 32$, single layer, last pooling). CMRU with $\epsilon = 0$ corresponds to BMRU. Increasing ϵ consistently improves mean accuracy and reduces variability. **Right:** Scaling across state dimensions $d \in \{4, 8, 16, 32, 64, 128, 256\}$ for CMRU ($\epsilon = 1$), α CMRU ($\epsilon = 1$), LRU, and minGRU (single layer). CMRU and α CMRU show more robust scaling with higher accuracy and lower variability, especially at smaller dimensions.

4.3. Persistent Memory under Continuous Noise

We next evaluate persistent memory and state efficiency directly using a copy-first-input task with continuous noisy inputs. A scalar value $x_0 \sim \mathcal{U}(-1, 1)$ is presented at $t = 0$ and must be reproduced at the final timestep after processing uniformly random noise $x_t \sim \mathcal{U}(-1, 1)$ for all $t > 0$. This design tests both long-term retention and the ability to reject irrelevant information. For the quantized CMRU, this task also reveals how efficiently the architecture exploits its discrete state space, by comparing its error against the theoretical quantization limit $\mathcal{E}_{\text{MAE}}^*$ (see Section C).

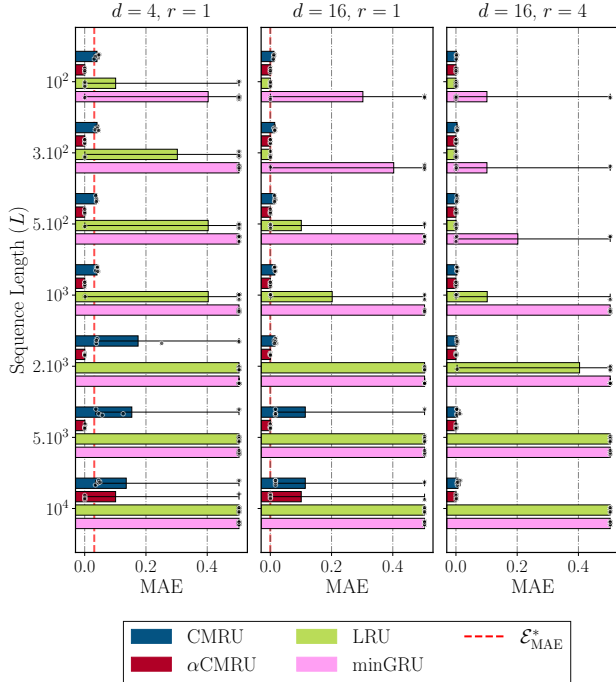


Figure 3. MAE on copy-first-input (continuous, noisy) versus sequence length L . **Left:** $r = 1, d = 4$. **Middle:** $r = 1, d = 16$. **Right:** $r = 4, d = 16$. Dashed red lines: CMRU quantization limits $\mathcal{E}_{\text{MAE}}^*$. Results shown as mean with min-max error bars over five seeds.

CMRU with $\epsilon = 1$ maintains consistent performance across sequence lengths L and capacity configurations (Figure 3). In the most constrained setting ($d = 4$, single layer and long sequence lengths), CMRU fails to solve the task robustly. This is due to limited noise rejection rather than gradient propagation issues, as these failures disappear in a noise-free version of the task (Figure 7, right). CMRU achieves MAE near the theoretical limit $\mathcal{E}_{\text{MAE}}^*$ at all sequence lengths, confirming that cumulative updates enable near-optimal memory encoding despite the discrete representation. We further validated this property on a 15-classes first-input classification task with $d = 4$: CMRU with $\epsilon = 1$ achieves 100% accuracy regardless of sequence length (Figure 7, left), showing optimal use of the hidden state.

The α CMRU with $\epsilon = 1$ removes the quantization constraint entirely, achieving near-perfect performance across all configurations. This confirms that input-dependent scaling offers a path to arbitrary precision when fixed quantization is insufficient.

In contrast, LRU and minGRU degrade substantially as sequence length increases, with high cross-seed variability, and in the presence or absence of noise. This is not merely an optimization issue: their dynamics fundamentally preclude perfect retention. LRU applies exponential decay through eigenvalues with magnitude less than one, causing stored information to fade continuously. minGRU lacks a true retention mode: even when the gate is near zero, continuous input processing prevents infinite memory preservation. At $L = 10000$, both architectures fail entirely. Even scaling to $r = 4$ layers with $d = 16$ does not consistently match a single-layer CMRU, confirming that the limitation is architectural, not capacity-related.

We further validate this advantage at larger scale. With $r = 6$ layers and $d = 256$, CMRU and α CMRU solve the task (continuous, noisy, $L = 5000$) robustly across all five seeds, while LRU and minGRU fail entirely. This confirms that the persistent memory advantage is not a small-model artifact (see Section H.3).

Summary and Choice of ε for Subsequent Experiments

Across all tasks and model capacities, increasing ε consistently improves convergence stability, reduces initialization sensitivity, and enhances final performance. The best results occur at $\varepsilon = 1$, where gradient magnitude equals unity in both retain and update modes. Based on these findings, we fix $\varepsilon = 1$ for all subsequent experiments.

5. Low-Capacity Scaling on Sequential Benchmarks

The preceding experiments establish that cumulative updates enable robust persistent memory. We now assess whether CMRU variants maintain competitive performance on tasks demanding more than long-term retention, and whether this competitiveness extends to practically relevant applications at minimal capacity.

5.1. Classical Sequential Benchmarks

We evaluate four benchmarks from the Long Range Arena suite (Tay et al., 2021): IMDB sentiment classification, sequential CIFAR-10, ListOps, and Pathfinder. These benchmarks span a range of computational requirements, from smooth temporal integration to compositional discrete reasoning and long-range spatial path-tracing. We employ mean pooling, following standard practice. We evaluate architectures with fixed state dimension $d = 32$ and varying depth $r \in \{1, 3, 6\}$ recurrent layers, except for Pathfinder where we use $r = 3$ only. Results are presented in Table 1.

Cumulative updates narrow the performance gap between CMRU and linear RNNs. On IMDB and sCIFAR10, LRU and minGRU achieve the highest accuracy, consistent with their fading memory mechanisms. However, the performance gap is small and narrows with increased depth. With $\varepsilon = 1$ the gradient magnitude equals unity along both the sequence axis; the CMRU thus acts as a residual connection in time, making deeper stacks as easy to train as shallower ones (Section B). We note that sCIFAR10 exhibited substantial overfitting across all architectures, a phenomenon also reported in prior work (Orvieto et al., 2023).

Discrete updates show advantages on ListOps. Both CMRU variants outperform the baselines at deeper architectures, achieving approximately 61% accuracy (close to state-of-the-art) with $r \geq 3$ layers, while LRU performance remains limited across all configurations. This suggests that persistent memory and discrete state transitions provide useful primitives for tasks requiring exact preservation of intermediate results.

Task-dependent architectural advantages emerge. Fading-memory architectures (LRU, minGRU) excel when information must be smoothly integrated over time, while

Table 1. Accuracy (%) on classical sequential benchmarks (IMDb, sCIFAR10, ListOps, Pathfinder). Results for different (r, d) shown as mean with min–max range. Pathfinder uses $(r, d) = (3, 32)$ only; chance level is 50%. **Dark green:** best performance; **medium green:** within 1%; **olive green:** within 2%.

(r, d)	α CMRU	CMRU	LRU	minGRU
IMDb				
(1, 32)	64.90 [61.00; 66.31]	64.84 [64.19; 65.44]	66.54 [64.16; 70.31]	65.88 [64.31; 67.28]
(3, 32)	64.62 [63.41; 65.56]	65.18 [64.34; 66.19]	66.84 [64.16; 70.25]	66.53 [64.38; 70.12]
(6, 32)	64.13 [60.19; 66.31]	65.04 [63.28; 66.75]	67.03 [61.47; 69.16]	66.21 [63.81; 67.69]
sCIFAR10				
(1, 32)	52.12 [51.34; 53.44]	56.27 [55.53; 57.31]	64.49 [63.06; 66.72]	58.64 [57.53; 59.31]
(3, 32)	58.64 [57.69; 60.09]	59.82 [55.25; 61.66]	59.99 [58.94; 61.44]	60.95 [60.44; 61.88]
(6, 32)	59.71 [57.69; 61.38]	58.81 [57.91; 59.59]	62.34 [58.75; 65.19]	62.12 [61.06; 62.84]
LISTOPS				
(1, 32)	38.74 [36.91; 39.75]	38.84 [36.84; 39.75]	38.64 [37.03; 39.75]	39.40 [37.69; 40.94]
(3, 32)	60.86 [59.94; 62.09]	60.42 [59.44; 61.03]	40.45 [37.75; 48.00]	55.07 [39.06; 60.28]
(6, 32)	61.36 [59.28; 62.94]	61.39 [60.38; 62.16]	39.37 [38.94; 39.91]	59.69 [57.59; 61.03]
PATHFINDER				
(3, 32)	92.00 [89.80; 93.70]	90.20 [87.30; 92.10]	50.14 [49.00; 51.50]	49.30 [48.10; 51.20]

persistent-memory architectures (CMRU, α CMRU) show strengths when information must be preserved through discrete operations. Neither family uniformly dominates. We discuss implications for architecture design in Section 7.

Persistent memory extends to harder long-range tasks. On Pathfinder, both CMRU variants achieve above 90% accuracy while LRU and minGRU perform at chance level ($\approx 50\%$), extending the pattern observed on ListOps. While larger fading-memory networks can solve Pathfinder with hyperparameter tuning (Orvieto et al., 2023), LRU and minGRU at this capacity ($r = 3, d = 32$) fail entirely, suggesting that persistent memory provides a decisive efficiency advantage for tasks requiring long-range state preservation.

Persistent memory complements fading memory in language modeling. We evaluate on character-level language modeling on the Shakespeare dataset, reporting cross-entropy loss on the test set (lower is better). Results are presented in Table 2. CMRU and α CMRU are competitive with minGRU and LRU, demonstrating that persistent memory does not impair performance on tasks dominated by smooth statistical structure. Hybrid architectures combining minGRU with CMRU or α CMRU layers achieve the best results, though the improvements are marginal at this scale. The primary finding here is one of compatibility: persistent and fading memory can be trained jointly

Table 2. Cross-entropy loss (lower is better) on Shakespeare character-level language modeling.

	α CMRU	CMRU	LRU	MINGRU	MINGRU +CMRU	MINGRU + α CMRU
LOSS	1.443 [1.440; 1.447]	1.455 [1.450; 1.461]	1.504 [1.496; 1.511]	1.453 [1.449; 1.458]	1.442 [1.438; 1.446]	1.441 [1.437; 1.445]

without conflict. Evidence that such combinations can yield substantial gains in learning efficiency and generalization comes from reinforcement learning, where combining both memory types leads to significant improvements in horizon generalization (Bakija et al., 2026). The CMRU and α CMRU results are obtained using ε -annealing during training; generalization to longer sequence lengths may require additional training strategies not explored in this work.

5.2. Audio Classification at Minimal Capacity

We now evaluate whether this competitiveness extends to *practically relevant* audio classification tasks at the *minimal capacity regime* required for ultra-low power deployment (Fyon et al., 2026a). We assess two keyword spotting (KWS) tasks derived from the Google Speech Commands dataset (Warden, 2018): digit recognition (11 classes) and full vocabulary classification (35 classes). These tasks represent canonical applications for *always-on* audio processing in battery-constrained devices. We evaluate single-layer architectures ($r = 1$) with state dimension $d \in \{4, 8, 16\}$, corresponding to easily implementable analog device sizes. Results are presented in Table 3. We extend the analysis to two-layer architectures and wake-word detection (binary: “yes” versus other words) in Section H.

All architectures achieve strong performance at minimal capacity. On KWS Digits, all cells exceed 95% accuracy even at $d = 4$. On KWS All (35 classes), performance ranges from 88% to 92%, with consistent improvement as state dimension increases.

 Table 3. Accuracy (%) for audio classification (KWS Digits, KWS All) with $r = 1$ layer and $d \in \{4, 8, 16\}$. Mean with min–max range over multiple initializations.

(r, d)	α CMRU	CMRU	LRU	MINGRU
KWS DIGITS				
(1, 4)	95.13 [94.75; 95.47]	95.12 [94.69; 95.50]	95.49 [94.62; 96.28]	95.92 [95.41; 96.31]
(1, 8)	95.09 [94.50; 95.69]	95.40 [95.03; 95.78]	96.16 [95.47; 96.56]	96.05 [95.81; 96.53]
(1, 16)	95.49 [95.22; 95.75]	95.38 [94.91; 95.91]	96.34 [95.88; 96.66]	95.99 [95.69; 96.25]
KWS ALL				
(1, 4)	88.41 [86.69; 90.00]	88.98 [88.34; 89.59]	89.85 [86.81; 91.09]	90.12 [87.41; 91.91]
(1, 8)	89.25 [88.19; 90.28]	88.99 [86.97; 90.03]	90.62 [88.81; 92.12]	91.82 [90.88; 92.88]
(1, 16)	89.53 [88.69; 90.28]	89.76 [88.97; 90.44]	91.82 [90.81; 92.88]	91.13 [90.84; 91.53]

Performance gaps are small and consistent. LRU and minGRU achieve marginally higher accuracy on most configurations, but cross-seed variability often exceeds inter-architecture differences.

CMRU enables ultra-low power deployment without sacrificing accuracy. CMRU achieves accuracy within 1–2% of the best-performing baselines while retaining compatibility with ultra-low power analog implementation. Its quantized structure maps directly to analog primitives, enabling 100 nW operation (Fyon et al., 2026a). A single-layer CMRU with $d = 16$ achieves 95.38% on digit recognition and 89.76% on full vocabulary classification.

6. Reflection Dynamics and Modular Computation

While the CMRU with $\varepsilon = 1$ demonstrates strong performance across tasks requiring persistent memory and long-range dependencies, we now identify a class of problems that expose fundamental limitations shared by all architectures examined in this work. Analyzing these limitations reveals promising directions for future work on extending the expressivity of parallelizable RNNs.

We evaluate all architectures on a binary parity classification task: given an input sequence $x = (x_1, x_2, \dots, x_L)$ where $x_t \in \{0, 1\}$, the model must predict $y = \left(\sum_{t=1}^L x_t\right) \bmod 2$. Models are trained on sequences of length $L \in [50, 400]$ and evaluated on $L \in [50, 1000]$ to assess length generalization, using last pooling with a single recurrent layer of dimension $d = 1$.

The CMRU variants ($\varepsilon \geq 0$) do not successfully solve this task, nor do the LRU or minGRU baselines (Table 8). For the CMRU with $\varepsilon = 1$, maintaining a cumulative sum is trivial. However, the modulo operation proves intractable for MLPs when generalizing to unseen sequence lengths, as the MLP must learn a periodic function over an unbounded domain. The root cause lies in the structure of reachable states: cumulative dynamics create countably infinite fixed points organized as a monotonic lattice, whereas the ideal solution for modular arithmetic requires states that alternate periodically. This mismatch suggests that different computational primitives are needed in the recurrent part to organize states in a way that makes the subsequent classification trivial.

Negative ε values enable the required reflection behavior. With $\varepsilon = -1$, the update equation becomes:

$$h_t = z_t \odot \left(S(\hat{h}_t) \odot \alpha - h_{t-1} \right) + (1 - z_t) \odot h_{t-1}. \quad (12)$$

During updates ($z_t = 1$), this implements a reflection operation that creates flip dynamics organizing the state space into alternating regions corresponding to even versus odd parity. This organization makes the subsequent modulo-2 classification trivial for the MLP, as the hidden state explicitly encodes parity through its position rather than its magnitude. Evaluating the CMRU with $\varepsilon = -1$ yields perfect accuracy (100%) with robust length generalization to $L = 1000$ (Table 8).

This finding directly aligns with recent work by [Grazzi et al. \(2025\)](#), which proves that state-tracking tasks (including parity) require transition matrices with eigenvalues having negative real parts. Our $\varepsilon = -1$ formulation instantiates this principle: as shown in Section B, the CMRU admits a selective SSM representation where ε directly controls the eigenvalues of the state transition matrix during updates. Setting $\varepsilon = -1$ yields eigenvalues at -1 , enabling the reflection dynamics necessary for modular computation.

7. Perspectives and Limitations

Complementarity between linear RNNs and CMRU computations. Our experiments reveal complementary strengths: fading-memory RNNs such as LRU and minGRU excel on tasks requiring smooth temporal integration, while CMRU variants show advantages on tasks requiring persistent memory and discrete state preservation. This specialization reflects fundamental differences in dynamics, which could be exploited through the development of hybrid architectures that combine the strengths of linear RNNs and CMRUs.

Toward ultra-low power RNN hardware. Two deployment paths are available. The first uses ε -annealing during training, producing models structurally identical to the original BMRU ($\varepsilon = 0$) that run directly on existing validated Schmitt trigger hardware ([Fyon et al., 2026a](#)) at sub-microwatt power levels; the annealing procedure is detailed in Section D. The second natively implements $\varepsilon = 1$ dynamics using a new analog circuit primitive. Circuit details, transistor-level schematics, and simulation results for both paths are provided in Section E.

The role of ε and learnable interpolation. Experiments have shown that the ideal value of ε is task-specific and connects to the role of eigenvalues in linear RNNs. Future work could focus on developing methods for automatic optimization of ε to avoid task-specific parameter tuning.

Limitations and future directions. Our evaluation focuses on relatively small model sizes motivated by ultra-low power deployment; scaling behavior at higher capacities and on more complex benchmarks remains an open question. We did not perform task-specific hyperparameter tuning, which may underestimate peak performance for all architectures: our consistent protocol prioritizes fair comparison and robustness over absolute benchmark numbers.

8. Conclusion

In this work, we establish persistent memory as a reliable, trainable primitive for parallelizable RNNs. We identified gradient blocking in update mode as a key limitation of the BMRU and introduced a cumulative update formulation that restores gradient flow while preserving persistent memory. The resulting CMRU and α CMRU show dramatically improved convergence stability, match fading-memory parallelizable RNNs on smooth integration benchmarks, and substantially outperform them on tasks requiring persistent memory: at minimal capacity, both variants exceed 90% accuracy on Pathfinder while LRU and minGRU remain at chance level. The CMRU retains the quantized states and discrete dynamics essential for ultra-low power analog implementation.

The CMRU is structurally complementary to fading-memory cells: where LRU, SSMs, and parallelizable gated RNNs excel at smooth integration, the CMRU excels at indefinite retention. This complementarity is a feature rather than a limitation. On character-level language modeling, hybrid architectures combining minGRU with CMRU or α CMRU layers achieve the best performance among the architectures we evaluated (Table 2), suggesting that pairing persistent and fading memory within the same network is a productive direction. More broadly, the CMRU is a general-purpose persistent-memory primitive: its associative-scan formulation and structural compatibility with existing SSM pipelines make it a drop-in persistent-memory block in Mamba-style architectures, complementing their fading-memory dynamics with the ability to retain discrete state indefinitely. Persistent memory is relevant wherever discrete state must be retained across arbitrary delays, independently of the power budget. Such memory is already a necessary condition for horizon generalization in reinforcement learning ([Bakija et al., 2026](#)), and we anticipate it will prove equally essential in other settings that demand long-range discrete recall.

Acknowledgements

J.B. and A.F. are, respectively, a FRIA grantee (FRIA-40038025) and a Research Fellow (ASP-REN40024838) of the Fonds de la Recherche Scientifique – FNRS. This work is supported by the Belgian Government through the Federal Public Service Policy and Support. Computational resources have been provided by the Consortium des Équipements de Calcul Intensif (CÉCI), funded by the FNRS under Grant No. 2.5020.11 and by the Walloon Region. The present research also benefited from computational resources made available on Lucia, the Tier-1 supercomputer of the Walloon Region, infrastructure funded by the Walloon Region under grant agreement No. 1910247. We thank Pierre Sacré, Loris Mendolia, and Asad Bakija for their valuable help, and the reviewers for their thoughtful and constructive discussions.

Disclosure

This work has been the subject of patent applications under numbers EP26175243.0 and EP26175248.9.

Impact Statement

The deployment of AI systems faces a dual challenge. First, the computational demands of modern deep learning contribute meaningfully to global energy consumption and carbon emissions. Second, high energy requirements prevent AI deployment in settings where power is severely constrained, leaving important applications unreachable despite their potential societal value. This work addresses both challenges by developing recurrent neural networks for ultra-low power deployment, achieving competitive performance at sub-microwatt power levels. By reducing energy consumption by three orders of magnitude compared to conventional implementations, this research both limits environmental impact and unlocks applications that were previously infeasible due to energy constraints. We believe that one of the levers research can provide is the study of specialized solutions that reduce energy consumption without sacrificing accuracy. By treating efficiency as a first-order design constraint rather than an afterthought, we can simultaneously reduce the environmental footprint of AI and expand its reach to energy-constrained settings where it could not otherwise be deployed. Reducing per-inference energy does not automatically ensure positive outcomes, and deployment contexts require ongoing attention from researchers, developers, and policymakers.

Software and Data

All code, raw training logs, tabulated results, and reproduction scripts are publicly available.² The implementation is provided in JAX (Bradbury et al., 2018; Heek et al., 2024), but the architectures and training procedures can be straightforwardly reproduced using any standard open-source machine learning library such as PyTorch or TensorFlow. The repository includes detailed instructions for reproducing all experiments presented in this paper.

All benchmark datasets used in this work are publicly available: MNIST (LeCun et al., 1998), CIFAR-10 (Krizhevsky & Hinton, 2009), IMDb (Maas et al., 2011), ListOps (Nangia & Bowman, 2018), Pathfinder (Tay et al., 2021), Shakespeare (McMahan et al., 2017), and Google Speech Commands (Warden, 2018). Synthetic tasks (copy first input, parity) are generated on-the-fly using the provided scripts.

References

- Arjovsky, M., Shah, A., and Bengio, Y. Unitary evolution recurrent neural networks. In *International Conference on Machine Learning*, volume 48, pp. 1120–1128. PMLR, 2016.
- Ba, J. L., Kiros, J. R., and Hinton, G. E. Layer normalization, 2016. URL <https://arxiv.org/abs/1607.06450>.
- Bakija, A., Geeter, F. D., Brandoit, J., Sacré, P., and Drion, G. On the importance of multistability for horizon generalization in reinforcement learning, 2026. URL <https://arxiv.org/abs/2605.12206>.
- Banbury, C., Reddi, V. J., Torelli, P., Holleman, J., Jeffries, N., Kiraly, C., Montino, P., Kanter, D., Ahmed, S., Pau, D., et al. MLPerf tiny benchmark. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.
- Beck, M., Pöppel, K., Spanring, M., Auer, A., Prudnikova, O., Kopp, M., Klambauer, G., Brandstetter, J., and Hochreiter, S. xLSTM: Extended long short-term memory. In *Advances in Neural Information Processing Systems*, volume 37, 2024.
- Bengio, Y., Simard, P., and Frasconi, P. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2):157–166, 1994. doi: 10.1109/72.279181.
- Billaudelle, S., Kriener, L., Moro, F., Torchet, T., and Payvand, M. MINIMALIST: switched-capacitor circuits for

²<https://github.com/julienbrandoit/ICML2026---Improving-Performance-and-Stability-of-Ultra-Low-Power-RNNs.git>

- efficient in-memory computation of gated recurrent units, 2025. URL <https://arxiv.org/abs/2505.08599>.
- Blelloch, G. E. Scans as primitive parallel operations. *IEEE Transactions on Computers*, 38(11):1526–1538, November 1989.
- Bouthillier, X., Delaunay, P., Bronzi, M., Trofimov, A., Nichyporuk, B., Szeto, J., Sepah, N., Raff, E., Madan, K., Voleti, V., Kahou, S. E., Michalski, V., Serdyuk, D., Arbel, T., Pal, C., Varoquaux, G., and Vincent, P. Accounting for variance in machine learning benchmarks. In *Proceedings of Machine Learning and Systems*, volume 3, pp. 747–769, 2021.
- Boyd, S. and Chua, L. O. Fading memory and the problem of approximating nonlinear operators with Volterra series. *IEEE Transactions on Circuits and Systems*, 32(11):1150–1161, 1985.
- Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., and Zhang, Q. JAX: composable transformations of Python+NumPy programs, 2018. URL <http://github.com/google/jax>.
- Chang, A. X. M., Martini, B., and Culurciello, E. Recurrent neural networks hardware implementation on FPGA, 2015. URL <https://arxiv.org/abs/1511.05552>.
- Cho, K., Van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., and Bengio, Y. Learning phrase representations using RNN encoder-decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 1724–1734, 2014.
- Dao, T. and Gu, A. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. In *International Conference on Machine Learning*, volume 235, pp. 10041–10071. PMLR, 2024.
- Dauphin, Y. N., Fan, A., Auli, M., and Grangier, D. Language modeling with gated convolutional networks. In *International Conference on Machine Learning*, volume 70, pp. 933–941. PMLR, 2017.
- Dayan, P. and Abbott, L. F. *Theoretical neuroscience: computational and mathematical modeling of neural systems*. MIT Press, 2005.
- Fedorov, I., Adams, R. P., Mattina, M., and Whatmough, P. N. SpArSe: Sparse architecture search for CNNs on resource-constrained microcontrollers. In *Advances in Neural Information Processing Systems*, volume 32, Vancouver, Canada, 2019.
- Feng, L., Tung, F., Hajimirsadeghi, H., Osama, M., Bengio, Y., and Memisevic, R. Were RNNs all we needed?, 2024. URL <https://arxiv.org/abs/2410.01201>.
- Fyon, A., Brandoit, J., Mendolia, L., Ernst, D., Redouté, J.-M., and Drion, G. Hardware-software co-design of scalable, energy-efficient analog recurrent computations, 2026a. URL <https://arxiv.org/abs/2605.15216>.
- Fyon, A., Mendolia, L., Redouté, J.-M., Franci, A., and Drion, G. A fully tunable ultra-low power current-mode memory cell in standard cmos technology, 2026b. URL <https://arxiv.org/abs/2605.07936>.
- Geeter, F. D., Lambrechts, G., Ernst, D., and Drion, G. Parallelizable memory recurrent units, 2026. URL <https://arxiv.org/abs/2601.09495>.
- Gray, R. M. and Neuhoff, D. L. Quantization. *IEEE Transactions on Information Theory*, 44(6):2325–2383, 1998. doi: 10.1109/18.720541.
- Grazzi, R., Siems, J., Zela, A., Franke, J. K. H., Hutter, F., and Pontil, M. Unlocking state-tracking in linear RNNs through negative eigenvalues. In *International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=ECGnSc3GNJ>.
- Gu, A. and Dao, T. Mamba: Linear-time sequence modeling with selective state spaces. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=tEYskw1VY2>.
- Gu, A., Dao, T., Ermon, S., Rudra, A., and Ré, C. HiPPO: Recurrent memory with optimal polynomial projections. In *Advances in Neural Information Processing Systems*, volume 33, 2020.
- Gu, A., Johnson, I., Goel, K., Saab, K., Dao, T., Rudra, A., and Ré, C. Combining recurrent, convolutional, and continuous-time models with linear state-space layers. In *Advances in Neural Information Processing Systems*, volume 34, 2021.
- Gu, A., Goel, K., Gupta, A., and Ré, C. On the parameterization and initialization of diagonal state space models. In *Advances in Neural Information Processing Systems*, volume 35, pp. 35971–35983, 2022a.
- Gu, A., Goel, K., and Ré, C. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations*, 2022b. URL <https://openreview.net/forum?id=uYLFoz1vlAC>.

- Gu, A., Johnson, I., Goel, K., Saab, K., Dao, T., Rudra, A., and Ré, C. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations*, 2022c. URL <https://openreview.net/forum?id=uYLFoz1vlAC>.
- Heek, J., Levskaya, A., Oliver, A., Ritter, M., Rondepierre, B., Steiner, A., and van Zee, M. Flax: A neural network library and ecosystem for JAX, 2024. URL <http://github.com/google/flax>.
- Helfrich, K., Willmott, D., and Ye, Q. Orthogonal recurrent neural networks with scaled Cayley transform. In *International Conference on Machine Learning*, volume 80, pp. 1969–1978. PMLR, 2018.
- Henderson, P., Islam, R., Bachman, P., Pineau, J., Precup, D., and Meger, D. Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018.
- Heydari, S. and Mahmoud, Q. H. Tiny machine learning and on-device inference: A survey of applications, challenges, and future directions. *Sensors*, 25(10):3191, 2025. doi: 10.3390/s25103191.
- Hochreiter, S. Untersuchungen zu dynamischen neuronalen Netzen. Diploma thesis, Technische Universität München, 1991.
- Hochreiter, S. and Schmidhuber, J. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- Hu, Y., Huber, A., Anumula, J., and Liu, S.-C. Overcoming the vanishing gradient problem in plain recurrent networks, 2019. URL <https://arxiv.org/abs/1801.06105>.
- Krizhevsky, A. and Hinton, G. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.
- Lambrechts, G., De Geeter, F., Vecoven, N., Ernst, D., and Drion, G. Warming up recurrent neural networks to maximise reachable multistability greatly improves learning. *Neural Networks*, 166:645–669, 2023.
- Le, Q. V., Jaitly, N., and Hinton, G. E. A simple way to initialize recurrent networks of rectified linear units, 2015. URL <https://arxiv.org/abs/1504.00941>.
- LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791.
- Lezcano-Casado, M. and Martínez-Rubio, D. Cheap orthogonality constraints in neural networks: A simple parametrization of the orthogonal and unitary group. In *International Conference on Machine Learning*, pp. 3794–3803. PMLR, 2019.
- Loshchilov, I. and Hutter, F. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=Bkg6RiCqY7>.
- Maas, A. L., Daly, R. E., Pham, P. T., Huang, D., Ng, A. Y., and Potts, C. Learning word vectors for sentiment analysis. In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, pp. 142–150, Portland, Oregon, USA, June 2011. Association for Computational Linguistics. URL <http://www.aclweb.org/anthology/P11-1015>.
- Martin, E. and Cundy, C. Parallelizing linear recurrent neural nets over sequence length, 2018. URL <https://arxiv.org/abs/1709.04057>.
- McFee, B., McVicar, M., Faronbi, D., Roman, I., Gover, M., Balke, S., Seyfarth, S., Malek, A., Raffel, C., Lostanlen, V., van Niekirk, B., Lee, D., Cwitkowitz, F., Zalkow, F., Nieto, O., Ellis, D., Mason, J., Lee, K., Steers, B., Halvachs, E., Thomé, C., Robert-Stöter, F., Bittner, R., Wei, Z., Weiss, A., Battenberg, E., Choi, K., Yamamoto, R., Carr, C., Metsai, A., Sullivan, S., Friesch, P., Krishnakumar, A., Hidaka, S., Kowalik, S., Keller, F., Mazur, D., Chabot-Leclerc, A., Hawthorne, C., Ramaprasad, C., Keum, M., Gomez, J., Monroe, W., Morozov, V. A., Eliasi, K., nullmightybofo, Biberstein, P., Sergin, N. D., Hennequin, R., Naktinis, R., beantowel, Kim, T., Åsen, J. P., Lim, J., Malins, A., Hereñú, D., van der Struijk, S., Nickel, L., Wu, J., Wang, Z., Gates, T., Vollrath, M., Sarroff, A., Xiao-Ming, Porter, A., Kranzler, S., VoodooHop, Gangi, M. D., Jinoz, H., Guerrero, C., Mazhar, A., tod-drme2178, Baratz, Z., Kostin, A., Zhuang, X., Lo, C. T., Campr, P., Semeniuc, E., Biswal, M., Moura, S., Brossier, P., Lee, H., Pimenta, W., Hyun, S., S. I., Rabinovich, E., Lei, G., Guo, J., Skelton, P. S. M., Pitkin, M., Mishra, A., Chaunin, S., BenedictSt, VanRavenswaay, S., and Südholt, D. librosa/librosa: 0.11.0, March 2025. URL <https://doi.org/10.5281/zenodo.15006942>.
- McMahan, B., Moore, E., Ramage, D., Hampson, S., and Arcas, B. A. y. Communication-Efficient Learning of Deep Networks from Decentralized Data. In Singh, A. and Zhu, J. (eds.), *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, volume 54 of *Proceedings of Machine Learning Research*, pp. 1273–1282. PMLR, 20–22 Apr 2017.

- Merrill, W., Petty, J., and Sabharwal, A. The illusion of state in state-space models. In *International Conference on Machine Learning*, volume 235. PMLR, 2024.
- Nangia, N. and Bowman, S. R. ListOps: A diagnostic dataset for latent tree learning. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Student Research Workshop*, pp. 92–99. Association for Computational Linguistics, 2018.
- Neftci, E. O., Mostafa, H., and Zenke, F. Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks. *IEEE Signal Processing Magazine*, 36(6):51–63, 2019.
- Orvieto, A., Smith, S. L., Gu, A., Fernando, A., Gulcehre, C., Pascanu, R., and De, S. Resurrecting recurrent neural networks for long sequences. In *International Conference on Machine Learning*, volume 202, pp. 26670–26698. PMLR, 2023.
- Pascanu, R., Mikolov, T., and Bengio, Y. On the difficulty of training recurrent neural networks. In *International Conference on Machine Learning*, volume 28, pp. 1310–1318. PMLR, 2013.
- Qin, Z., Yang, S., and Zhong, Y. Hierarchically gated recurrent neural network for sequence modeling. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- Smith, J. T. H., Warrington, A., and Linderman, S. W. Simplified state space layers for sequence modeling. In *International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=Ai8Hw3AXqks>.
- Talathi, S. S. and Vartak, A. Improving performance of recurrent neural network with ReLU nonlinearity, 2015. URL <https://arxiv.org/abs/1511.03771>.
- Tay, Y., Dehghani, M., Abnar, S., Shen, Y., Bahri, D., Pham, P., Rao, J., Yang, L., Ruder, S., and Metzler, D. Long range arena: A benchmark for efficient transformers. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=qVyeW-grC2k>.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pp. 5998–6008, 2017.
- Vecoven, N., Ernst, D., and Drion, G. A bio-inspired bistable recurrent cell allows for long-lasting memory. *PLoS ONE*, 16(6):e0252676, 2021.
- Vorontsov, E., Trabelsi, C., Kadoury, S., and Pal, C. On orthogonality and learning recurrent networks with long term dependencies. In *International Conference on Machine Learning*, volume 70, pp. 3570–3578. PMLR, 2017.
- Warden, P. Speech commands: A dataset for limited-vocabulary speech recognition, 2018. URL <https://arxiv.org/abs/1804.03209>.
- Warden, P. and Situnayake, D. *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O’Reilly Media, 2019.
- Wisdom, S., Powers, T., Hershey, J. R., Roux, J. L., and Atlas, L. Full-capacity unitary recurrent neural networks. In *Advances in Neural Information Processing Systems*, volume 29, 2016.
- Zhu, R.-J., Zhang, Y., Abreu, S., Sifferman, E., Sheaves, T., Wang, Y., Richmond, D., Shrestha, S. B., Zhou, P., and Eshraghian, J. K. Scalable MatMul-free language modeling, 2024. URL <https://arxiv.org/abs/2406.02528>.

A. Related Work

Parallelizable recurrent architectures. The renewed interest in RNNs for sequence processing, particularly as alternatives to Transformers (Vaswani et al., 2017), can largely be attributed to recent developments in structured state-space models (SSMs) (Gu et al., 2022c;a). The Mamba architecture (Gu & Dao, 2024) and its successor Mamba-2 (Dao & Gu, 2024) have demonstrated that parallelizable recurrent models can achieve competitive performance with Transformers on many benchmarks. A fundamental theoretical gap exists between parallelizable architectures and classical nonlinear RNNs: recent work (Merrill et al., 2024) has shown that both Transformers and linear diagonal SSMs are constrained to the complexity class TC^0 , while classical RNNs can express problems in NC^1 , which strictly contains TC^0 under widely held complexity-theoretic conjectures. Importantly, our experiments demonstrate that even within TC^0 (the class of problems both Transformers and classical SSMs can theoretically express), significant practical performance differences persist among parallelizable recurrent cells. We show that persistent memory capabilities, which enable robust gradient propagation and perfect information preservation, are key for improving performance on long-range tasks. Parallelizable gated RNNs have emerged as an alternative to SSMs, including xLSTM (Beck et al., 2024), HGRN (Qin et al., 2023), MatMul-free LM (Zhu et al., 2024), and GILR (Martin & Cundy, 2018). We adopt the minimal GRU (minGRU) (Feng et al., 2024) as our representative baseline due to its simplicity as one of the most minimal fully parallelizable gated architectures.

Gradient propagation and memory in RNNs. In recurrent formulations where processing is inherently local, in contrast to attention mechanisms that enable global information flow, the implementation of persistent memory remains challenging. Persistent memory is intrinsically linked to gradient quality, which directly impacts both long-range modeling capabilities and training stability. While Transformers largely avoid vanishing and exploding gradient issues through their architecture, the return to recurrent formulations reintroduces these fundamental challenges (Hochreiter, 1991; Bengio et al., 1994). The dynamical systems perspective on these phenomena (Pascanu et al., 2013) provides valuable insights into the mechanisms underlying gradient pathologies.

Fading versus persistent memory. From a dynamical systems perspective, the distinction between fading memory and persistent memory is well-established (Boyd & Chua, 1985). Linear dynamics imply monostability: states decay exponentially toward a unique equilibrium, which is the signature of fading memory. True persistent memory requires multistability, which linear dynamics cannot exhibit.

However, multistability is rarely discussed in the machine learning literature despite its fundamental importance for implementing persistent memory. Notable exceptions include recent work exploring multistability in neural systems (Vencoven et al., 2021; Lambrechts et al., 2023) and the original BMRU formulation (Geeter et al., 2026).

Unitary and orthogonal RNNs. The desirability of eigenvalues near or at unit magnitude for stable gradient propagation has long been recognized (Pascanu et al., 2013). One line of work focuses on initialization strategies that place eigenvalues near the unit circle across different timescales (Le et al., 2015; Gu et al., 2020; Talathi & Vartak, 2015), often grounded in theoretical frameworks such as HiPPO theory (Gu et al., 2020). The Linear Recurrent Unit (LRU) (Orvieto et al., 2023), which we adopt as our primary baseline, exemplifies this approach with its exponentially parameterized eigenvalues initialized within $[r_{\min}, r_{\max}]$. An alternative approach maintains unitary eigenvalues as a structural constraint during optimization (Arjovsky et al., 2016; Wisdom et al., 2016). Unitary transition matrices can be constructed through products of simple unitary blocks or by optimizing directly on the Stiefel manifold. Related approaches employ the Cayley transform (Helfrich et al., 2018) and enforce orthogonality through soft or hard constraints (Vorontsov et al., 2017; Lezcano-Casado & Martínez-Rubio, 2019). However, in these approaches, the transition matrices are learned but remain fixed with respect to the input during inference. Our CMRU formulation with $\varepsilon = 1$ achieves unit eigenvalues structurally in both retain and update modes (see Section B), combining the benefits of unitary RNNs with the flexibility of modern selective mechanisms.

Input-dependent (selective) mechanisms. Modern SSMs have introduced the concept of selectivity, where model parameters vary as functions of the input (Gu & Dao, 2024). This input-dependence enables the model to selectively filter and retain information based on context, significantly improving performance on tasks requiring context-aware processing. Our work builds upon this paradigm: as shown in Section B, the CMRU admits a selective SSM representation where the state transition matrix $A(x_t)$ depends on the input through the gate z_t , enabling discrete switching between memory modes while maintaining exactly unitary eigenvalues during retention. Recent theoretical work by Grazi et al. (2025) has demonstrated that state-tracking tasks (including parity) require transition matrices with eigenvalues having negative real parts. Our reflection dynamics with $\varepsilon = -1$ (Section 6) directly instantiate this principle, achieving perfect parity classification where all other configurations fail.

Gating mechanisms and non-differentiable activations.

Classical gating mechanisms in LSTMs (Hochreiter & Schmidhuber, 1997) and GRUs (Cho et al., 2014), as well as modern parallelizable variants such as minGRU (Feng et al., 2024), employ gates constrained to the open interval $(0, 1)$ through smooth sigmoid activations. Critically, these gates can only asymptotically approach the boundary values 0 and 1, never reaching them exactly. This design choice, likely stemming from the historical requirement for differentiable non-linearities, fundamentally limits the ability to implement truly persistent memory, as gates cannot achieve the exact values necessary for perfect information preservation. The CMRU employs the Heaviside step function and sign function to achieve exact binary gating. To enable gradient-based learning with these discontinuous activations, we draw on surrogate gradient methodology developed in the spiking neural networks community (Neftci et al., 2019). Surrogate gradients substitute a smooth approximation during the backward pass while maintaining the discrete operation during the forward pass, enabling us to leverage exact binary switching while maintaining trainability through standard backpropagation.

Cumulative and integrator dynamics. The concept of integrator dynamics in recurrent systems has received limited attention in the machine learning literature, despite its importance for tasks requiring unbounded accumulation. Classical leaky integrators from neuroscience (Dayan & Abbott, 2005) implement exponential decay combined with input integration, but suffer from fading memory limitations. Perfect integrators, characterized by eigenvalues of exactly unity, are theoretically capable of unbounded accumulation without decay. Our CMRU formulation with $\varepsilon = 1$ provides a mechanism for implementing selective integration through a structural constraint that enforces unitary eigenvalues during both retain and update modes. The resulting cell accumulates quantized inputs into the hidden state while maintaining perfect gradient flow ($\frac{\partial h_t}{\partial h_{t-1}} = 1$ in both modes), enabling robust learning of long-range dependencies.

Ultra-low power neural network hardware. The deployment of neural networks on resource-constrained devices has motivated significant research in efficient architectures (Banbury et al., 2021; Warden & Situnayake, 2019). Most existing implementations, whether on microcontrollers or FPGAs, emulate neural computation in digital hardware (Fedorov et al., 2019; Chang et al., 2015). True analog implementations offer fundamentally different scaling properties: latency independent of network depth and energy consumption decoupled from arithmetic precision. The BMRU was specifically designed for hardware-software co-design: its bistable, quantized structure maps directly to Schmitt triggers, a well-characterized analog primitive

requiring only transistors. Recent implementations have demonstrated $>90\%$ accuracy at 100 nW power consumption for keyword spotting (Fyon et al., 2026b;a), three orders of magnitude below digital alternatives. The CMRU with $\varepsilon = 1$ requires different circuit primitives (e.g., gated charge accumulators) but maintains noise resilience through discrete accumulated values. The ε -annealing strategy described in Section D provides a path to deploy gradient-friendly training while retaining compatibility with existing Schmitt trigger hardware.

Recent work has also explored the hardware realization of parallelizable gated RNNs such as the minGRU in low power regimes. In particular, Billaudelle et al. (2025) propose a mixed-signal implementation of a modified minGRU using switched-capacitor in-memory computing. Their design relies on aggressive quantization, discrete-time operation, and explicit clocking to orchestrate charge transfer, gating, and state updates. While this approach demonstrates impressive energy efficiency compared to fully digital accelerators, its energy consumption and latency remain fundamentally tied to the clock frequency and the required sequencing of sampling, sharing, and analog-to-digital conversion phases.

B. The α CMRU Admits a Selective State-Space Representation

We demonstrate that the CMRU and α CMRU cells admit a selective state-space representation with unique properties that enable robust persistent memory. This result applies to both the CMRU with fixed quantization scale α and its generalization, the α CMRU with input-dependent scaling $\alpha(x_t)$.

Consider a non-linear state-space model (SSM) (or in the modern language of SSMs, a *selective SSM*) defined by

$$h_t = A(x_t)h_{t-1} + B(x_t)x_t, \quad (13)$$

where $h_t \in \mathbb{R}^d$ is the hidden state, $A(x_t) \in \mathbb{R}^{d \times d}$ denotes the state transition matrix and $B(x_t) \in \mathbb{R}^{d \times m}$ denotes the input-to-state mapping, with both matrices parameterized as functions of the input $x_t \in \mathbb{R}^m$. Throughout, we treat all vectors as column vectors, so that $x_t^T \in \mathbb{R}^{1 \times m}$ denotes the corresponding row vectors. While this model is nonlinear due to the input-dependence of A and B , it remains linear with respect to the hidden state h_t , which permits the application of parallel scan algorithms for efficient computation of hidden states. Representative examples of such models include Mamba (Gu & Dao, 2024) and Mamba-2 (Dao & Gu, 2024).

A fundamental limitation of SSMs concerns the implementation of persistent memory as it would require the eigenvalues of the state transition matrix A to be on the unitary circle.

Any deviation from this condition results in either vanishing or exploding memory over extended sequences, with the magnitude of deviation scaling as $\lambda^{\Delta t^*}$, where λ denotes an eigenvalue and $\Delta t^* \in \mathbb{N}$ represents the number of time steps elapsed since the last informative input.

Proposition B.1. *The CMRU and α CMRU cells can be expressed as a non-linear SSM wherein the state transition matrix A maintains unitary eigenvalues during the retain mode, thereby enabling robust persistent memory.*

Proof. We rewrite the CMRU (and α CMRU) update equation as

$$h_t = \text{diag}(1 - z_t + \varepsilon z_t) h_{t-1} + z_t \odot S_\alpha(\hat{h}_t), \quad (14)$$

where $\text{diag}(\cdot)$ constructs a square diagonal matrix from its vector argument, and where $S_\alpha(\cdot)$ is a short notation for $\alpha \odot S(\cdot)$ or $\alpha_t \odot S(\cdot)$, such that we handle both the CMRU and the α CMRU through a single notation. The input-driven term can be decomposed as $z_t \odot S_\alpha(\hat{h}_t) = B(x_t)x_t$. We note that $B(x_t)$ is not uniquely defined. Indeed, consider the general form

$$B(x_t) = B_0(x_t) + C, \quad (15)$$

where $C \in \mathbb{R}^{d \times m}$ is an arbitrary matrix whose rows lie in the null space of x_t (i.e., $Cx_t = 0$), and

$$B_0(x_t) = (z_t \odot S_\alpha(\hat{h}_t)) \frac{x_t^T}{x_t^T x_t}. \quad (16)$$

Then,

$$\begin{aligned} (B_0(x_t) + C)x_t &= B_0(x_t)x_t + Cx_t \\ &= B_0(x_t)x_t \\ &= (z_t \odot S_\alpha(\hat{h}_t)) \frac{x_t^T}{x_t^T x_t} x_t \\ &= (z_t \odot S_\alpha(\hat{h}_t)) \frac{x_t^T x_t}{x_t^T x_t} \\ &= z_t \odot S_\alpha(\hat{h}_t), \end{aligned} \quad (17)$$

establishing the desired decomposition. We adopt the canonical form with $C = 0$, yielding the following non-linear SSM representation:

$$A(x_t) = \text{diag}(1 - z_t + \varepsilon z_t), \quad (18)$$

$$B(x_t) = (z_t \odot S_\alpha(\hat{h}_t)) \frac{x_t^T}{x_t^T x_t}. \quad (19)$$

This representation holds for both the CMRU (with fixed quantization scale α) and the α CMRU (with input-dependent $\alpha(x_t)$), as the scaling parameter is absorbed into $S_\alpha(\hat{h}_t)$ within the $B(x_t)$ matrix. $\square$

Remark B.2. During the retain mode, characterized by $z_t = 0$, the state transition matrix reduces to the identity:

$$A(x_t)|_{z_t=0} = I, \quad (20)$$

which possesses unit eigenvalues exclusively. This interpretation of the CMRU and α CMRU as a special case of non-linear SSM with unitary eigenvalues during the retain mode provides a theoretical justification for their ability to model long range dependencies effectively. Unitary eigenvalues combined with the hysteresis non-linearity ensure that *information can be preserved indefinitely and robustly in the hidden state*. The robustness is ensured by the hysteretic window, which acts as a noise filter that prevents small perturbations from spuriously triggering gate transitions and corrupting the stored memory.

During the update mode, characterized by $z_t = 1$, the state transition matrix reduces to the diagonal matrix:

$$A(x_t)|_{z_t=1} = \text{diag}(\varepsilon). \quad (21)$$

Interestingly, it means that for $\varepsilon = 1$, the state transition matrix reduces to identity for both the retain and the update mode. This has an important consequence for deep architectures: gradient magnitude equals unity simultaneously along the sequence axis (through time) and along the depth axis (across stacked layers), since each recurrent block contributes a unit Jacobian in both modes. The CMRU with $\varepsilon = 1$ therefore acts as a residual connection in time, making training of deep stacks as stable as training shallow ones. We can also observe that the parity example (Section 6), requiring at least one negative eigenvalue, can be directly related to the recent work of (Grazzi et al., 2025) under the matrix-vector transition formalization.

B.1. Fixed Points in Retain Mode

In retain mode ($z_t = 0$), setting $z_t = 0$ in Equation (7) gives $h_t = h_{t-1}$ directly. Every $h \in \mathbb{R}^d$ is therefore a fixed point of the update equation, for any value of ε : the CMRU retains any state indefinitely in the absence of a sufficiently strong input, which is the formal expression of persistent memory. The hysteretic threshold β_t controls how strong an input must be to exit retain mode and trigger an update.

C. Optimal Quantization on $[-1, 1]$ and Minimal MAE

C.1. Theoretical Derivations

We consider the problem of encoding a scalar real value $x_0 \in [-1, 1]$ using $b = 4$ binary memory slots, and we derive the minimal achievable mean absolute error (MAE) under an optimal fixed-rate quantization scheme. This encoding problem is associated with the *copy first input (continuous)* tasks. While this result is well known in classical

quantization theory, we present it here to establish the absolute lower bound of error that any quantized learning cell can achieve, thereby providing a fundamental limit against which to evaluate the (quantized) BMRU.

For a uniform source distribution on a bounded interval, the optimal scalar quantizer (for both MAE and MSE) is a uniform quantizer with midpoint reconstruction (Gray & Neuhoff, 1998). With $B = 2^b$ quantization levels, the interval $[-1, 1]$ is partitioned into bins of equal width:

$$\Delta = \frac{2}{B}. \quad (22)$$

Each bin is associated with its midpoint as the reconstruction value. The quantization error $e(x_0) = \hat{x}_0 - x_0$ therefore lies in the interval $[-\Delta/2, \Delta/2]$. Conditioned on a given bin, $e(x_0)$ is uniformly distributed on this interval, and its expected absolute value is:

$$\mathbb{E}[|e(x_0)|] = \frac{1}{\Delta} \int_{-\Delta/2}^{\Delta/2} |u| du = \frac{\Delta}{4}. \quad (23)$$

Since all bins have equal probability under the uniform distribution, this value is also the global MAE. Substituting Δ yields:

$$\mathcal{E}_{\text{MAE}}^* = \frac{1}{2^{b+1}}. \quad (24)$$

Although the interval is symmetric around zero and thus includes signed values, the sign of x_0 does not require a dedicated bit. The sign is implicitly encoded by the quantization index itself, as the quantization levels are distributed across both negative and positive values.

C.2. Extensions to the Cumulative Variants

At first glance, one might expect the optimal quantization bound $\mathcal{E}_{\text{MAE}}^*$ to not apply to the cumulative variants of the BMRU (i.e., when $\varepsilon \neq 0$). Indeed, the cumulative formulation allows the hidden state to accumulate values over multiple timesteps, potentially enabling the state to be pushed toward any multiple of the quantization level α . This suggests that, in principle, a CMRU ($\varepsilon \neq 0$) could integrate information over time to progressively refine its encoding and achieve representations that transcend the discrete 2^b levels available when $\varepsilon = 0$.

However, two properties of our specific task and architecture prevent this theoretical advantage from being realized in practice:

(1) State-independent updates. In all the recurrent cells we consider, the relative update to the hidden state at each timestep is computed independently of the previous state value. This architectural constraint means that the cell cannot adaptively modulate its updates based on the current encoding error or state value with a single-block architecture.

(2) Single-timestep information presentation. In the copy first input task, the information to be encoded is presented at a single timestep ($t = 0$). This structure eliminates any opportunity for the cell to integrate information across multiple timesteps to progressively refine its approximation.

These two conditions together ensure that, despite the theoretical flexibility of cumulative state updates when $\varepsilon \neq 0$, the CMRU remains subject to the same fundamental quantization constraint.

D. Training with ε Annealing for Hardware Deployment

Our experiments establish that $\varepsilon = 1$ provides superior gradient flow, convergence stability, and reduced initialization sensitivity. However, existing ultra-low power analog implementations are designed for $\varepsilon = 0$ (the original BMRU), where the architecture maps directly onto Schmitt trigger circuits (Fyon et al., 2026a). The CMRU with $\varepsilon = 1$ requires alternative circuit primitives such as gated charge accumulators. We explore an intermediate path: training with $\varepsilon = 1$ to leverage optimal gradient propagation, then gradually decaying ε toward zero to produce models compatible with existing hardware.

Annealing protocol. We interpolate ε from 1 to 0 during training according to a piecewise linear schedule: ε remains at 1 for the first 5% of training, decays linearly to 0 over the next 70%, then stays at 0 for the final 25%. This stationary phase addresses the theoretical concern that time-varying ε renders the optimization problem non-stationary. We only save checkpoints after $\varepsilon = 0$ is reached, ensuring reported performance reflects the deployable configuration.

Preliminary results. Table 4 presents accuracy on sMNIST across state dimensions $d \in \{4, 8, 16, 32, 64, 128, 256\}$ for three regimes: constant $\varepsilon = 0$ (i.e., the original BMRU), constant $\varepsilon = 1$, and decaying ε . For the quantized CMRU, annealing substantially improves over constant $\varepsilon = 0$ (88% versus 30% at $d = 32$) but does not fully recover constant $\varepsilon = 1$ performance (96% at $d = 32$). The gap is most pronounced at small dimensions where discrete state constraints dominate. For the α CMRU, annealing closely matches constant $\varepsilon = 1$ across nearly all dimensions (97% versus 97% at $d = 32$, 98% versus 97% at $d = 64$), suggesting continuous states successfully exploit gradient flow during training even when deployment requires $\varepsilon = 0$ dynamics.

These results demonstrate that ε annealing provides a viable path to deploy gradient-friendly training on existing ultra-low power hardware, though quantized architectures at minimal capacity may require ε -aware circuit (or α -aware circuit) implementations to fully realize performance gains.

Table 4. Accuracy (%) for sMNIST with decaying epsilon across state dimensions. Note that $\varepsilon = 0$ corresponds to the original BMRU.

	STATE DIMENSION						
	4	8	16	32	64	128	256
CMRU	30.29	51.68	79.12	87.94	95.46	96.78	97.03
DECAY ε	[20.94; 47.72]	[32.41; 62.34]	[66.75; 85.19]	[81.88; 92.53]	[95.00; 96.44]	[96.31; 97.22]	[96.62; 97.56]
CMRU	83.61	91.49	94.83	95.92	97.05	96.20	96.16
$\varepsilon = 1$	[80.97; 85.09]	[90.53; 92.69]	[93.12; 95.84]	[95.41; 96.19]	[96.69; 97.28]	[95.16; 97.00]	[95.81; 96.38]
CMRU	14.52	17.09	21.76	35.32	47.49	54.82	80.11
$\varepsilon = 0$	[11.69; 18.62]	[11.69; 21.41]	[18.69; 25.09]	[28.12; 44.41]	[46.16; 49.38]	[48.62; 73.69]	[76.09; 85.34]
α CMRU	88.13	93.82	96.12	96.94	97.73	98.09	98.07
DECAY ε	[87.28; 89.41]	[93.22; 94.59]	[95.59; 96.88]	[96.22; 97.28]	[97.69; 97.84]	[97.88; 98.25]	[97.88; 98.25]
α CMRU	88.34	93.49	95.43	96.76	96.88	97.16	97.25
$\varepsilon = 1$	[87.38; 89.53]	[91.56; 94.62]	[93.03; 96.47]	[96.22; 97.00]	[96.66; 97.12]	[96.72; 97.91]	[96.69; 98.06]
α CMRU	32.79	40.78	44.76	48.86	69.60	88.30	94.06
$\varepsilon = 0$	[24.62; 42.94]	[38.38; 42.75]	[38.56; 55.16]	[43.12; 64.84]	[66.12; 75.69]	[83.78; 94.59]	[92.06; 95.28]

E. Hardware Implementation

E.1. Deployment via ε -Annealing on Existing Hardware

The ε -annealing strategy (detailed in Section D) trains with $\varepsilon = 1$ for optimal gradient flow, then decays ε linearly to 0. The resulting model is structurally identical to the original BMRU and deploys on existing Schmitt trigger circuits without any new circuit primitives, achieving sub-microwatt operation as validated in [Fyon et al. \(2026a\)](#).

E.2. Proof-of-Concept Native CMRU Circuit

The native CMRU with $\varepsilon = 1$ requires a different analog primitive. We propose a cascade of BMRU cells interleaved with Differential Pair Integrator (DPI) blocks. Each DPI integrates toward α when the corresponding BMRU output is high, and the hysteresis windows are shifted by α at each stage to implement progressive triggering. The output is the sum of all BMRU outputs, implementing the cumulative state update. All parameters are tunable via bias currents, and both BMRU and DPI blocks operate in the subthreshold regime.

Transistor-level schematics and Cadence simulations (Figure 4) confirm ultra-low power operation at 39.86 nW. Circuit area, power, and mismatch robustness optimization are left for future work.

F. Architecture and Training Details

F.1. Backbone Architecture Details

All experiments are performed by only changing the type of recurrent cell in the backbone illustrated in Figure 5. In all experiments, the model dimension is fixed to $m = 256$. The model dimension m refers to the dimensionality of sequences between layers, while the state dimension d refers to the dimensionality of the recurrent cell hidden state, and r to the number of blocks. Note that m and d are independent: the recurrent cell projects from m to d

internally, and projects back to m for the output.

F.2. Cell Output and Integration

Each cell defines an internal state transition law that maps the input sequence to a hidden state sequence $h_{1:T} \in \mathbb{R}^{d \times T}$ via $h_{1:T} = \text{Cell}(x_{1:T}; h_0)$, where h_0 represents the initial state and d denotes the state dimension. All architectures considered in this work are parallelizable through associative scan algorithms ([Blelloch, 1989](#); [Martin & Cundy, 2018](#); [Smith et al., 2023](#)), enabling efficient computation of hidden states across the entire sequence.

From the hidden sequence, the output sequence is generated through an input-gated projection of the normalized hidden sequence:

$$\begin{aligned} y_{1:T} &= \widetilde{\text{Cell}}(x_{1:T}) \\ &= \text{Norm}(\text{Linear}(\text{Cell}(x_{1:T}; h_0))) \\ &\quad \odot \sigma(\text{Linear}(x_{1:T})), \end{aligned} \quad (25)$$

where $\sigma(\cdot)$ is the element-wise sigmoid function, $\text{Linear}(\cdot)$ denotes a learned linear transformation, and $\text{Norm}(\cdot)$ represents layer normalization. The normalization ensures that even integrator cells ($\varepsilon = 1$) output sequences whose magnitudes do not explode, maintaining proper scale for subsequent MLPs and blocks. The input-dependent gate $\sigma(\text{Linear}(x_{1:T}))$ ensures that each cell can learn to decouple information integration from information usage over the sequence. The complete block described by Equation (25) constitutes a sub-layer $\text{SubLayer}(\cdot)$ that is integrated into the broader backbone architecture.

F.3. MLP and Normalization Layers

We use $\sigma(\cdot) = \text{GLU}(\cdot)$ ([Dauphin et al., 2017](#)) within each MLP. The MLPs are defined as

$$\text{MLP}(x) = \text{Linear}(\text{Dropout}(\sigma(\text{Linear}(x)))) ,$$

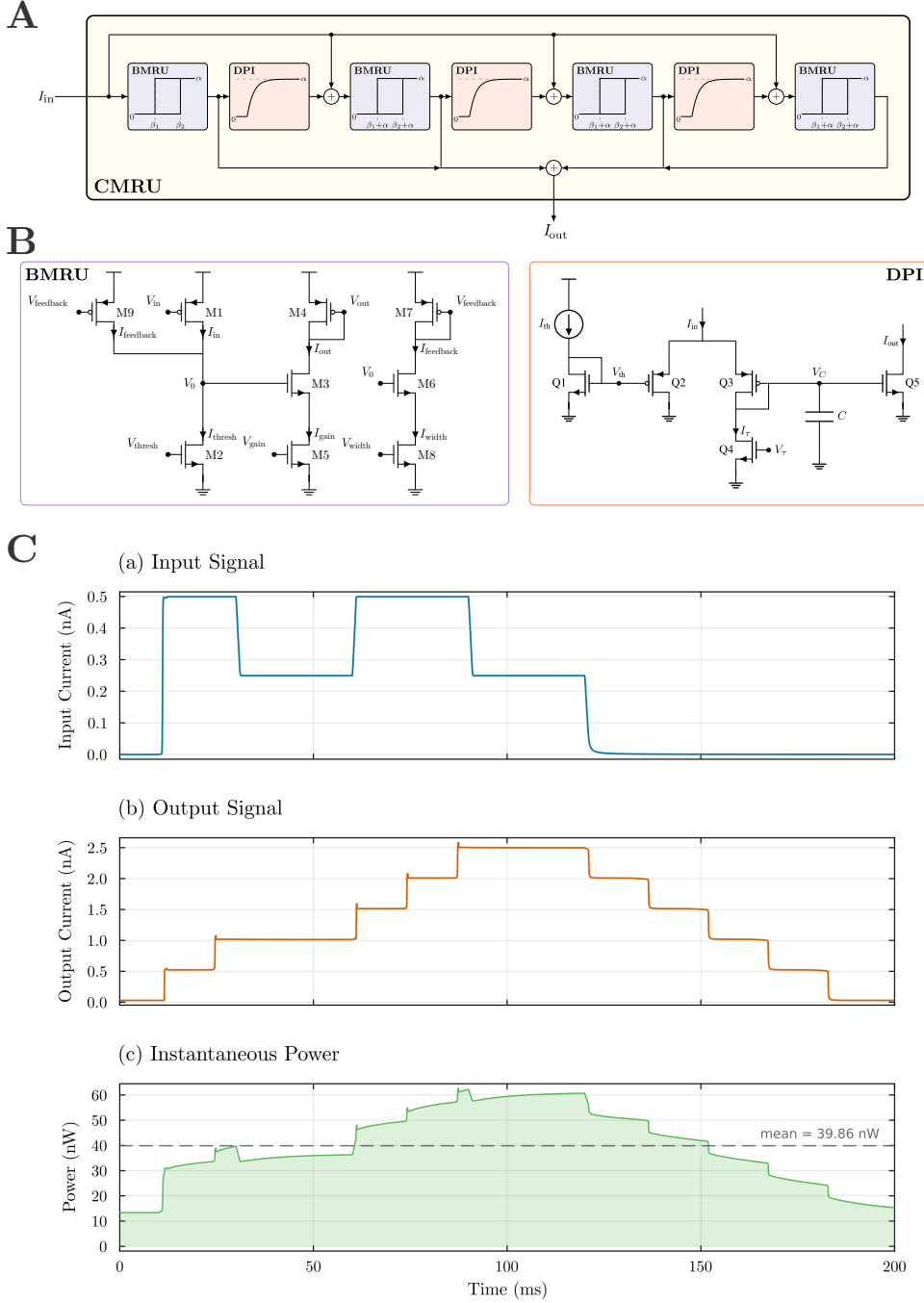


Figure 4. Native CMRU analog circuit. **(A)** Block diagram of the conceptual architecture. The CMRU is realized as a cascade of BMRU cells (Fyon et al., 2026a) interleaved with Differential Pair Integrator (DPI) blocks. Each BMRU stage occupies one level of the fixed-point ladder, and each DPI integrates the output of the preceding stage toward a multiple of α , shifting the hysteresis window of the next stage. The finite set of reachable fixed points, located at integer multiples of α , matches the discrete cumulative state values that naturally emerge from training. **(B)** Transistor-level schematics of the BMRU cell (left) and of the DPI block (right). Both blocks operate in the subthreshold regime. **(C)** Transistor-level Cadence simulation results. The high threshold of the BMRUs is set to 0.35 nA, the low threshold to 0.15 nA, α to 0.5 nA, and the DPI time constant to 10 ms. Top: time evolution of the input signal. Depending on amplitude, the input either drives the memory to one of its two states (high or low) or, at the retain level of 0.25 nA, leaves the stored value unchanged. Middle: time evolution of the output signal. In set mode, the output is incremented (or decremented) by α every 10 ms; in retain mode, the output is held indefinitely. Bottom: time evolution of the instantaneous power. Consumption scales with the output level but remains deep in the ultra-low-power regime, well below the microwatt range.

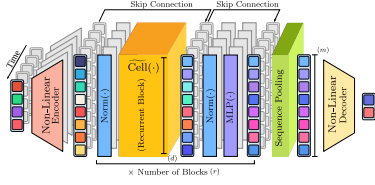


Figure 5. Common backbone architecture used across all experiments. The architecture interleaves recurrent layers with MLPs, skip connections, and normalization. We study different recurrent cell types (CMRU, α CMRU, LRU, minGRU) by varying the internal mechanism within $\widetilde{\text{Cell}}(\cdot)$ while keeping the backbone structure fixed. The model dimension is fixed at $m = 256$ across all experiments. The state dimension d refers to the dimensionality of the recurrent cell hidden state, and r to the number of blocks.

with the input and output dimension equal to the model dimension ($m = 256$) and a hidden dimension of $4 \times m$. The MLPs are applied point-wise, i.e., independently on each element in the sequence.

Normalization and skip-connection layers are applied point-wise and following a pre-norm scheme, i.e.,

$$y = v \odot x + \text{SubLayer}(\text{Norm}(x)),$$

where $v \in \mathbb{R}^m$ is a vector of learnable parameters, initialized to unity. The normalization used is LayerNorm (Ba et al., 2016). In this formulation, $\text{SubLayer}(\cdot)$ can be $\text{MLP}(\cdot)$ or $\widetilde{\text{Cell}}(\cdot)$ as described in Equation (25).

F.4. Sequence Pooling

The backbone architecture must reduce the variable-length processed sequence to a fixed-size representation for prediction. We employ two pooling strategies depending on the task requirements:

Last pooling uses only the final timestep of the processed sequence for prediction. Formally, given the output sequence $y_{1:T} \in \mathbb{R}^{m \times T}$ from the final recurrent layer, the pooled representation is $y_{\text{pool}} = y_T \in \mathbb{R}^m$. This strategy is particularly suited for tasks requiring long-term memory, as it tests both the retention of information across the entire sequence and the ability to learn from long-range dependencies when supervision occurs only at sequence end.

Mean pooling averages the output sequence across all timesteps. The pooled representation is computed as $y_{\text{pool}} = \frac{1}{T} \sum_{t=1}^T y_t \in \mathbb{R}^m$. This strategy aggregates information from the entire sequence, making it more robust to local variations and suitable for tasks where relevant information is distributed throughout the sequence rather than concentrated at specific positions.

The pooled representation y_{pool} is then passed through a final decoder to produce task-specific predictions.

F.5. Positional Encoding

All recurrent cells receive explicit positional information through sinusoidal positional encoding (PE) (Vaswani et al., 2017). The PE is concatenated to the input at each timestep and passed through a learned linear projection before being fed to the recurrent cell. Formally, for input $x_t \in \mathbb{R}^m$ at position t , the cell receives:

$$\tilde{x}_t = \text{Linear}([x_t; \text{PE}(t)]) \in \mathbb{R}^m, \quad (26)$$

where $\text{PE}(t) \in \mathbb{R}^{d_{\text{PE}}}$ denotes the sinusoidal positional encoding at position t , and $[\cdot; \cdot]$ denotes concatenation.

This design allows architectures that do not inherently require positional information (such as the LRU) to learn to suppress it through the projection if it acts as harmful noise, while enabling cells that benefit from explicit position awareness to leverage it when beneficial. Positional encoding has been shown to be necessary for the original BMRU with $\varepsilon = 0$ (Geeter et al., 2026), where the lack of inherent temporal dynamics makes position information critical for sequence processing, but also for single-block minGRU architectures (Feng et al., 2024).

F.6. Input Encoding and Output Decoding

Raw task-specific inputs and outputs are transformed to and from the model dimension m through nonlinear encoding and decoding layers.

The input encoder transforms task-specific input $\hat{x}_t \in \mathbb{R}^{d_{\text{task}}}$ to the model dimension:

$$x_t = \tilde{x}_t + \text{MLP}(\tilde{x}_t), \quad \text{where } \tilde{x}_t = \text{Linear}(\hat{x}_t). \quad (27)$$

Similarly, the output decoder transforms from model dimension back to task-specific output dimension d_{out} :

$$\hat{y} = \tilde{y} + \text{MLP}(\tilde{y}), \quad \text{where } \tilde{y} = \text{Linear}(y_{\text{pool}}), \quad (28)$$

where $y_{\text{pool}} \in \mathbb{R}^m$ is the pooled representation.

This nonlinear encoding and decoding design ensures that single-layer recurrent architectures are not limited by insufficient point-wise nonlinear computation capacity, but rather by the recurrent mechanism itself. This allows for fair architectural comparisons where observed performance differences can be attributed to the recurrent cell design rather than to inadequate input/output transformations.

F.7. Recurrent Cell Implementations

This section provides detailed specifications of the recurrent cells evaluated in our experiments. For the CMRU and α CMRU architectures, we refer to Equations (2) to (4), (7) and (10) in the main text.

F.7.1. LINEAR RECURRENT UNIT (LRU)

The LRU (Orvieto et al., 2023) implements a diagonal complex-valued state-space model with strictly decaying exponentially parameterized eigenvalues for numerical stability. The state update follows:

$$x_t = \Lambda \odot x_{t-1} + Bu_t, \quad (29)$$

$$y_t = \text{Re}(Cx_t) + Du_t, \quad (30)$$

where $x_t \in \mathbb{C}^d$ is the complex-valued hidden state, $u_t \in \mathbb{R}^m$ is the input, $y_t \in \mathbb{R}^d$ is the output (before projection), and $\Lambda \in \mathbb{C}^d$ is a diagonal matrix of eigenvalues. The eigenvalues are parameterized as:

$$\Lambda = \exp(-\exp(\nu) + i \exp(\theta)), \quad (31)$$

where $\nu, \theta \in \mathbb{R}^d$ are learned parameters initialized such that eigenvalue magnitudes lie in $[r_{\min}, r_{\max}]$ with uniformly distributed phases in $[0, 2\pi]$. The input matrix $B \in \mathbb{C}^{m \times d}$ is normalized by $\gamma = \sqrt{1 - |\Lambda|^2}$ to maintain consistent input scaling across different eigenvalue magnitudes. The output matrices $C \in \mathbb{C}^{d \times d}$ and $D \in \mathbb{R}^{m \times d}$ are learned parameters.

The LRU is fully parallelizable via associative scan over the linear recurrence.

F.7.2. MINIMAL GATED RECURRENT UNIT (MINGRU)

The minGRU (Feng et al., 2024) simplifies the traditional GRU by removing hidden state dependencies from the gating mechanism, enabling full parallelization while maintaining competitive performance. The update rule is:

$$z_t = \sigma(W_z x_t + b_z), \quad (32)$$

$$\tilde{h}_t = W_h x_t + b_h, \quad (33)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t, \quad (34)$$

where $h_t \in \mathbb{R}^d$ is the hidden state, $x_t \in \mathbb{R}^m$ is the input, $z_t \in [0, 1]^d$ is the update gate, $\tilde{h}_t \in \mathbb{R}^d$ is the candidate hidden state, and $\sigma(\cdot)$ denotes the sigmoid function.

A critical architectural property is that the sigmoid gate ensures $z_t \in (0, 1)$ (open interval), meaning the values $z_t = 0$ and $z_t = 1$ are only approached asymptotically but never attained. Consequently, the minGRU lacks a true retain mode where $z_t = 0$ would preserve the state unchanged, and lacks a true reset mode where $z_t = 1$ would completely overwrite the state. At every timestep, both the previous state and the candidate contribute to the update, preventing indefinite information preservation.

The update equation is a linear recurrence in h_t and can be parallelized via associative scan.

F.8. Training Generalities

All experiments presented in this work follow a similar training procedure with hyperparameters that are not optimized per task. The objective is to observe (and encourage) the robustness of the cells to the use of standard hyperparameter values, while acknowledging that hyperparameter optimization could improve absolute performance metrics. We refer readers to Section 3.4 for the motivations.

Optimization is performed with AdamW (Loshchilov & Hutter, 2019) with $\beta_1 = 0.9$, $\beta_2 = 0.99$ and $\epsilon = 10^{-8}$ for all experiments. The weight decay (WD) value used is 0.0001. The learning rate is set to 10^{-3} and follows a common scheduler across all experiments: (1) a linear warmup phase over 1% of iterations starting from 0 up to the value of 10^{-3} ; followed by (2) a cosine decay to the minimum value of 10^{-5} spread over all remaining iterations. The gradient norm is clipped to 1 during training.

We adopt the terminology of iterations or, equivalently, epochs in the sense of a single gradient descent step throughout this work. The maximum number of epochs varies from task to task to ensure convergence within the limits of a reasonable computational budget (see Table 5). For each task, the current architecture is evaluated every 64 iterations on 20 batches sampled from the validation set. The checkpoint achieving the best performance during this evaluation is the one saved and used during the final evaluation procedure on the test set. Early stopping is performed if the performance is optimal during 100 consecutive evaluations (that is, 100% accuracy). A batch size of 64 is used.

We use cross-entropy loss for classification tasks and mean-squared error loss for regression tasks.

Regarding the hyperparameters specific to the type of recurrent cells used, no optimization was performed. For the LRU, the initialization of eigenvalues follows the procedure described by (Orvieto et al., 2023), with $[r_{\min}; r_{\max}] = [0.9; 0.999]$ (which seems to be a good compromise between hyperparameter optimization and an initialization favorable to learning long-range dependencies) and a phase in $[0; 2\pi]$.

For the CMRU and α CMRU cells, we remove the particular initialization used in (Geeter et al., 2026) which aimed to bias the cell toward a retain mode ($z_t = 0$) at the start of training. This trick was necessary to tackle the gradient blocking phenomenon when $z_t = 1$, but is no longer necessary thanks to the introduction of ϵ (see Section 3.1). We do not rule out that a particular initialization could accelerate the convergence of results; we acknowledge that a statistical approach could provide insight into optimal initialization values. The surrogate used in our implementation follows the original implementation $\frac{dH(x)}{dx} \stackrel{\text{surr}}{\approx} \frac{1}{1 + (\pi \alpha_{\text{surr}} x)^2}$, and we use $\alpha_{\text{surr}} = 1$.

Table 5. Maximum number of training iterations (epochs) allocated per task. All tasks use a batch size of 64, with evaluation performed every 64 iterations on 20 validation batches. Early stopping is triggered if optimal performance (100% accuracy for classification tasks) is maintained for 100 consecutive evaluations.

TASK	COPY FIRST	SMNIST	PMNIST	SCIFAR -10	LISTOPS	PATH FINDER	LM	IMDB	KWS	PARITY
MAX ITER.	100,000	30,000	30,000	100,000	100,000	100,000	100,000	100,000	35,000	35,000

G. Benchmark Task Descriptions

This appendix provides detailed descriptions of all benchmark tasks used throughout this work. For each task, we specify the input-output structure, evaluation metric, and dataset characteristics. All tasks are evaluated using the protocol described in Section 3.4.

G.1. Sequential Image Classification Tasks

G.1.1. SEQUENTIAL MNIST (sMNIST)

Sequential MNIST presents handwritten digit images (28×28 pixels) as sequences of 784 grayscale pixel values in raster scan order (left-to-right, top-to-bottom). The task requires classifying the digit (0–9) based solely on the sequential pixel stream.

Task structure. Each input sequence $\{x_t\}_{t=1}^{784}$ consists of normalized pixel intensities $x_t \in [0, 1]$. The model processes the sequence and produces a classification decision at the final timestep. Ground truth labels $y \in \{0, 1, \dots, 9\}$ provide supervision only at sequence end. We use last pooling for this task.

Dataset. We use the standard MNIST training set (60,000 samples) and test set (10,000 samples) (LeCun et al., 1998). A validation set is created by withholding 10,000 samples from the training data.

Evaluation metric. Classification accuracy on the test set.

G.1.2. PERMUTED MNIST (PMNIST)

Permuted MNIST applies a fixed random permutation to the pixel ordering of MNIST images, destroying spatial correlations while preserving information content. Conventionally regarded as more challenging than sMNIST, we challenge this assumption through evaluations presented in Section H.1.

Task structure. Given a fixed permutation $\pi : \{1, \dots, 784\} \rightarrow \{1, \dots, 784\}$, each image is presented as the sequence $\{x_{\pi(t)}\}_{t=1}^{784}$. The permutation is identical across all samples and remains fixed during training and evaluation.

Variants. We evaluate three different random permutations: (1) pMNIST; (2) pMNIST₂₄; and (3) pMNIST₃₆.

Dataset and evaluation. Identical to sMNIST.

G.1.3. SEQUENTIAL CIFAR-10 (sCIFAR10)

Sequential CIFAR-10 extends sequential image classification to natural color images with 10 object categories. Following (Orvieto et al., 2023), we use the colored version of the task and mean pooling. This task is one of those proposed by (Tay et al., 2021) to challenge Transformer architectures.

Task structure. Color images ($32 \times 32 \times 3$) are flattened into sequences of length 1,024 by concatenating pixels in raster scan order. Each timestep presents a single pixel value $x_t \in [0, 1]^3$ after normalization.

Dataset. CIFAR-10 training set (50,000 samples) and test set (10,000 samples), with 10% of training data reserved for validation (Krizhevsky & Hinton, 2009).

Evaluation metric. Classification accuracy on the test set.

G.2. Long-Term Memory Tasks

All long-term memory tasks use last pooling, where supervision is provided only at the final timestep, jointly assessing memory retention and gradient propagation over the entire sequence length.

G.2.1. COPY FIRST INPUT (DISCRETE, 15 ITEMS)

This classification task isolates the ability to encode and retain discrete symbolic information over arbitrary sequence lengths.

Task structure. At timestep $t = 0$, the model observes a one-hot encoded input from a vocabulary of 15 classes. All subsequent timesteps ($t > 0$) present zero vectors. At the final timestep $t = L$, the model must classify the original input.

Sequence lengths. We evaluate $L \in \{100, 300, 500, 1000, 2000, 5000, 10000\}$.

Dataset. Generated synthetically with 10,000 training samples, 2,000 validation samples, and 2,000 test samples per sequence length. Class labels are uniformly distributed.

Evaluation metric. Classification accuracy on the test set. Random-guess baseline is 6.67%.

G.2.2. COPY FIRST INPUT (CONTINUOUS, NO NOISE)

This regression task requires memorizing and reproducing a continuous scalar value with no confounding factors.

Task structure. At timestep $t = 0$, the model observes a scalar value $x_0 \sim \mathcal{U}(-1, 1)$. All subsequent timesteps present $x_t = 0$. The model must reproduce x_0 at the final timestep $t = L$.

Sequence lengths and dataset. Identical to the discrete variant.

Evaluation metric. Mean absolute error (MAE) on the test set. For quantized architectures, we compare against the theoretical quantization limit $\mathcal{E}_{\text{MAE}}^*$ derived in Section C.

G.2.3. COPY FIRST INPUT (CONTINUOUS, NOISY)

This regression task extends the continuous variant by requiring robust memory retention in the presence of continuous noise.

Task structure. At timestep $t = 0$, the model observes $x_0 \sim \mathcal{U}(-1, 1)$. For all $t > 0$, the model observes uniformly random noise $x_t \sim \mathcal{U}(-1, 1)$. The model must reproduce x_0 at timestep $t = L$ despite continuous distractor inputs.

Sequence lengths and dataset. Identical to previous variants.

Evaluation metric. Mean absolute error (MAE) on the test set.

G.2.4. COPY FIRST INPUT AT LARGE SCALE (CONTINUOUS, NOISY)

This experiment extends the copy-first-input (continuous, noisy) variant to a larger network to verify that the persistent memory advantage is not limited to the minimal-capacity regime.

Task structure. Identical to the copy-first-input (continuous, noisy) task: a scalar $x_0 \sim \mathcal{U}(-1, 1)$ is presented at $t = 0$, followed by uniformly random noise $x_t \sim \mathcal{U}(-1, 1)$ for all $t > 0$. The model must reproduce x_0 at the final timestep.

Network configuration. We evaluate $r = 6$ recurrent layers with $d = 256$. This is substantially larger than the minimal-capacity experiments in the main text.

Sequence length. $L = 5000$.

Evaluation metric. Mean absolute error (MAE) on the test set.

G.3. Natural Language Processing Tasks

G.3.1. IMDB SENTIMENT CLASSIFICATION

IMDb sentiment classification requires determining whether a movie review expresses positive or negative sentiment. Following the Long Range Arena benchmark (Tay et al., 2021), we use byte-level encoding and mean pooling.

Task structure. Movie reviews are encoded at the byte level, with each byte mapped to a learned embedding. Reviews vary in length with maximum sequence length capped at 4,000. The model processes the byte sequence with mean pooling to produce a binary classification (positive/negative).

Dataset. IMDb Large Movie Review Dataset (Maas et al., 2011) containing 50,000 reviews. We used 40,000 for training, 10,000 for validation, and the rest as the test dataset.

Evaluation metric. Classification accuracy on the test set. Random-guess baseline is 50%.

G.3.2. LISTOPS

ListOps evaluates compositional reasoning over hierarchically structured sequences of operations (Tay et al., 2021).

Task structure. Sequences represent nested operations (MAX, MIN, MEDIAN, SUM_MOD) applied to integer arguments. Sequences are linearized in prefix notation with special tokens marking brackets and operations. Each token is mapped to a learned embedding, and the model processes sequences with mean pooling to produce the final numerical result.

Dataset. We use the standard ListOps dataset (Nangia & Bowman, 2018) with sequences up to length 2,000. The training set consists of 96,000 samples, while both the validation and test sets contain 2,000 samples each.

Evaluation metric. Classification accuracy on the discrete output values (10 possible output classes).

G.3.3. PATHFINDER

Pathfinder is a visual long-range dependency task from the Long Range Arena benchmark (Tay et al., 2021). The task requires determining whether a dashed path connects two circles in a 32×32 grayscale image.

Task structure. Images are flattened into sequences of length $L = 1,024$ pixels. The model processes the pixel sequence and produces a binary classification (connected / not connected) using mean pooling.

Dataset. We use the standard LRA Pathfinder split with 160,000 training samples, 20,000 validation samples, and 20,000 test samples.

Evaluation metric. Classification accuracy on the test set. Random-guess baseline is 50%.

G.3.4. CHARACTER-LEVEL LANGUAGE MODELING (SHAKESPEARE)

Character-level language modeling on the Shakespeare corpus requires predicting the next character given all preceding characters.

Task structure. The corpus is encoded at the character level. At each timestep, the model observes a character token and must predict the next character. We use the full sequence without pooling, computing the loss at every timestep.

Dataset. The complete works of Shakespeare, split into training, validation, and test sets by contiguous segments.

Evaluation metric. Cross-entropy loss on the test set.

G.4. Audio Classification Tasks

All audio tasks use the Google Speech Commands dataset (Warden, 2018), which contains one-second audio clips of 35 spoken words sampled at 16 kHz. Audio is preprocessed using Mel-frequency cepstral coefficients (MFCCs), producing sequences of 101 timesteps with 13 features per timestep. We rely on `librosa` for feature extraction (McFee et al., 2025). For all subtasks, we construct balanced datasets with 80% of samples used for training, 10% for validation, and 10% for testing. We use mean pooling.

G.4.1. KEYWORD SPOTTING: DIGITS (KWS DIGITS)

Task structure. 11-way classification distinguishing spoken digits (“zero” through “nine”) and background noise.

Evaluation metric. Classification accuracy on the test set. Random-guess baseline is 9.09%.

G.4.2. KEYWORD SPOTTING: YES VERSUS OTHERS (KWS YES VERSUS OTHERS)

Task structure. Binary classification detecting the word “yes” against a set of negative words: “background noise”, “no”, “up”, “down”, “left”, “right”. This represents a wake-word detection scenario.

Evaluation metric. Classification accuracy on the test set. Random-guess baseline is 50%.

G.4.3. KEYWORD SPOTTING: ALL CLASSES (KWS ALL)

Task structure. 35-way classification across the full Speech Commands vocabulary.

Evaluation metric. Classification accuracy on the test set. Random-guess baseline is 2.86%.

G.5. Modular Arithmetic Tasks

G.5.1. PARITY

The parity task requires computing the XOR of all binary inputs in a sequence, equivalent to determining whether the total count of ones is even or odd.

Task structure. Input sequences $\{x_t\}_{t=1}^L$ consist of binary values $x_t \in \{0, 1\}$. The target output $y \in \{0, 1\}$ is computed as $y = \left(\sum_{t=1}^L x_t\right) \bmod 2$. We use last pooling, with supervision provided only at the final timestep.

Dataset. Generated synthetically on-the-fly with binary inputs drawn uniformly at random. Models are trained and validated on sequences of length $L \in [50, 400]$, then evaluated on sequences of length $L \in [50, 1000]$ to assess length generalization.

Evaluation metric. Classification accuracy on the test set. Random-guess baseline is 50%. Length generalization is critical for considering the task genuinely solved.

H. Additional Experimental Results

This appendix presents supplementary experimental results that extend and support the findings in the main text.

H.1. Permuted MNIST: Reconsidering the Difficulty Assumption

Permuted MNIST (pMNIST) has been conventionally regarded as a more challenging benchmark than Sequential MNIST (sMNIST). This view is rooted in the observation that the fixed random permutation of pixels destroys spatial

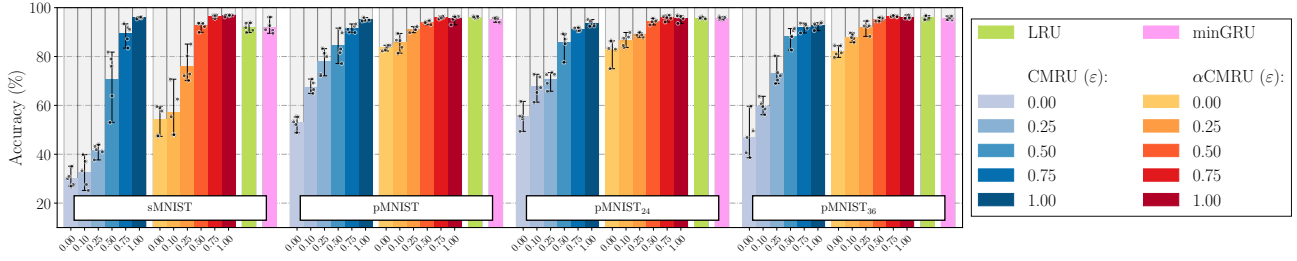


Figure 6. Accuracy on the pixel-by-pixel MNIST classification task across different pixel orderings. Each architecture is evaluated over five random initializations (shown as scattered points), with results reported as mean across seeds with min-max error bars. Cell types are presented from left to right within each task as: CMRU with $\epsilon \in \{0.00, 0.10, 0.25, 0.50, 0.75, 1.00\}$, then α CMRU for the same ϵ values, followed by LRU and minGRU. Note that $\epsilon = 0$ corresponds to the original BMRU. The four panels show results for: sMNIST (sequential, raster scan order), pMNIST (permutation seed 42), pMNIST₂₄ (permutation seed 24), and pMNIST₃₆ (permutation seed 36). Sequential MNIST proves consistently harder across all architectures, with gradient-robust models ($\epsilon = 1$) demonstrating the largest performance advantage. All models use state dimension $d = 32$ and a single recurrent layer. Random-guess accuracy is 10%.

correlations, forcing models to discover hidden relationships between pixels rather than exploiting natural image structure (Le et al., 2015; Hu et al., 2019). While this reasoning captures an important aspect of the task complexity, we argue that it presents an overly simplistic view of difficulty in sequence modeling benchmarks when dealing with recurrent architectures.

The conventional perspective conflates two distinct notions of difficulty: the inherent complexity of the solution and the difficulty of discovering that solution during training. A problem may be deemed “harder” not because it requires a more sophisticated solution, but because gradient-based optimization struggles to find any viable solution at all. In the context of recurrent architectures susceptible to vanishing gradients, this distinction becomes critical.

Sequential MNIST presents a specific challenge that exacerbates vanishing gradient problems: MNIST digits typically contain substantial whitespace and are centered, resulting in long sequences of near-zero (black) pixels, particularly toward the end of each sequence when presented in raster scan order. These extended uninformative regions create pathways through which gradients must propagate during back-propagation through time. In contrast, permuted MNIST distributes pixels more uniformly throughout the sequence, potentially providing more frequent gradient signals and mitigating the vanishing gradient problem.

To test this hypothesis, we evaluated our models on sMNIST and three different random permutations (pMNIST, pMNIST₂₄, and pMNIST₃₆, corresponding to random seeds 42, 24, and 36 respectively). The results, presented in Figure 6, yield two key observations:

Sequential MNIST is harder than permuted variants. Across all cell types tested, the sequential version consistently achieves lower accuracy than any of the three permuted versions. Our results suggest that the natural raster

scan ordering is more challenging than a typical random permutation, supporting our hypothesis that the distribution of informative content throughout the sequence significantly impacts learnability for gradient-based methods.

Architectures robust to vanishing gradients excel on the harder task. Both the α CMRU and the CMRU with $\epsilon = 1$ achieve performance equivalent to or better than the LRU and minGRU on the permuted variants. Critically, these gradient-robust architectures demonstrate substantially superior performance on the harder sMNIST task. Furthermore, increasing ϵ toward 1 consistently improves performance while dramatically reducing initialization-induced variability across all tasks, in agreement with our theoretical gradient analysis.

H.2. Copy First Input: Discrete and Clean Continuous Variants

The copy first input task provides a direct assessment of a model’s ability to maintain persistent memory over extended sequences. In this task, a single informative signal appears at $t = 0$, followed by uninformative inputs (zeros), and the model must reproduce the initial signal at the final timestep after a delay of length L .

We evaluate two complementary variants of this task, both presented in Figure 7:

Copy first input (discrete, 15 items). This classification task requires the model to memorize a discrete symbol from a vocabulary of 15 classes. With sequence lengths $L \in \{100, 300, 500, 1000, 2000, 5000, 10000\}$, this task evaluates the model’s ability to implement robust multi-stable memory states. For the CMRU with state dimension $d = 4$, this vocabulary size represents critical capacity.

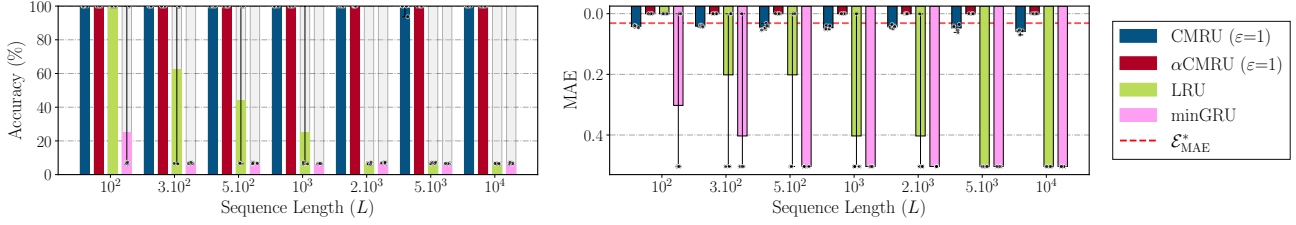


Figure 7. Performance on copy first input tasks as a function of sequence length L . Each architecture is evaluated over five random initializations (shown as scattered points), with results reported as mean across seeds with min-max error bars. Cell types are presented from left to right within each sequence length as: CMRU, α CMRU (both with $\varepsilon = 1$), LRU, and minGRU. All models use hidden state dimension $d = 4$ and a single recurrent layer. **Left:** Discrete vocabulary task (15 classes), accuracy for $L \in \{100, 300, 500, 1000, 2000, 5000, 10000\}$. Random-guess accuracy is $1/15 \approx 6.7\%$. **Right:** Continuous regression task (clean, noise-free). MAE with y-axis inverted (higher bars = better). The dashed red line $\varepsilon_{\text{MAE}}^*$ indicates the quantization limit for the CMRU at $d = 4$; this bound does not apply to LRU, minGRU, or α CMRU.

Copy first input (continuous, clean). This regression task requires the model to memorize a continuous scalar value $x_0 \sim \mathcal{U}(-1, 1)$ presented at the first timestep and reproduce it at the final timestep. This variant contains only zeros after $t = 0$, eliminating confounding effects from noise filtering.

The results reveal stark differences in how architectures handle these memory-intensive tasks. Both the CMRU and α CMRU with $\varepsilon = 1$ demonstrate robust performance that remains stable as sequence length increases. In contrast, the LRU and minGRU exhibit substantial performance degradation as sequences lengthen. We argue that this degradation reflects an optimization challenge rather than an inherent architectural limitation: individual seeds that converge successfully achieve near-perfect performance, demonstrating sufficient representational capacity. The difficulty lies in reliably discovering these solutions during training across long sequences where gradients must propagate through the entire sequence length.

H.3. Copy First Input at Large Scale

We evaluate all architectures on the copy-first-input (continuous, noisy) task at a substantially larger scale than the main text experiments: $r = 6$ recurrent layers and $d = 256$, with sequence length $L = 5000$. The task setup is described in Section G.2.4.

CMRU and α CMRU solve the task robustly across all five seeds. LRU and minGRU fail **entirely** across all seeds, yielding MAE near the trivial baseline. This result rules out the possibility that the persistent memory advantage observed at small model sizes is a consequence of insufficient representational capacity in the fading-memory baselines: the architectural distinction between persistent and fading memory determines success or failure on this task.

H.4. Extended Audio Classification Results

We present additional audio classification results not included in the main text: performance on the wake-word detection task (KWS Yes versus Others) and results extended to two-layer configurations ($r = 2$).

Table 6. Accuracy (%) for KWS Yes versus Others with $r = 1$ recurrent layer. Results shown as mean with min-max range over multiple initializations.

(r, d)	α CMRU	CMRU	LRU	MINGRU
	98.12	98.12	98.18	97.89
(1, 4)	[97.34; 98.56]	[97.59; 98.75]	[97.47; 98.59]	[97.03; 98.66]
	97.96	98.25	98.41	98.34
(1, 8)	[97.50; 98.19]	[97.84; 98.66]	[97.81; 99.00]	[97.88; 99.12]
	97.61	98.30	98.45	98.26
(1, 16)	[96.97; 98.03]	[97.88; 98.62]	[98.22; 99.06]	[97.56; 99.00]

On KWS Yes versus Others, all architectures achieve approximately 98% accuracy even at $d = 4$, with differences falling within cross-seed variability. This confirms that single-layer CMRU architectures are sufficient for wake-word detection, achieving 98.30% with $d = 16$ while maintaining compatibility with 100 nW analog implementations (Fyon et al., 2026a).

Comparison with single-layer results. Comparing Table 7 with Table 3 reveals modest improvements from adding a second recurrent layer. On KWS Yes versus Others, all architectures achieve approximately 98% accuracy with both $r = 1$ and $r = 2$, indicating that a single layer suffices for this binary detection task. On KWS Digits and KWS All, the additional layer provides marginal gains (typically 0.3–0.5%).

Implications for deployment. These results support the use of single-layer CMRU architectures for ultra-low power keyword spotting applications. The marginal accuracy gain from $r = 2$ (typically $< 0.5\%$) does not justify the increased circuit complexity and power consumption for analog im-

Table 7. Accuracy (%) for audio classification tasks (KWS Yes versus Others, KWS Digits, KWS All) with $r = 2$ recurrent layers. Results shown as mean with min–max range over multiple initializations.

(r, d)	α CMRU	CMRU	LRU	MINGRU
KWS YES VERSUS OTHERS				
	97.78	98.45	98.37	98.16
(2, 4)	[96.53; 98.34]	[97.53; 98.91]	[97.62; 99.00]	[97.31; 98.72]
	98.28	98.38	98.27	98.49
(2, 8)	[97.88; 98.66]	[97.84; 98.97]	[97.47; 98.81]	[97.66; 98.91]
	97.67	98.35	98.65	98.40
(2, 16)	[96.81; 98.44]	[97.72; 98.75]	[98.19; 99.06]	[97.66; 99.03]
KWS DIGITS				
	95.44	95.43	95.94	96.06
(2, 4)	[94.81; 95.97]	[95.16; 95.72]	[95.00; 96.91]	[95.53; 96.44]
	95.94	95.34	96.12	96.19
(2, 8)	[95.69; 96.31]	[94.91; 95.69]	[95.34; 96.53]	[95.88; 96.53]
	95.79	95.82	96.43	96.34
(2, 16)	[94.97; 96.91]	[95.44; 96.16]	[95.91; 96.84]	[96.03; 96.59]
KWS ALL				
	89.97	89.38	90.38	91.38
(2, 4)	[88.75; 90.94]	[87.91; 90.50]	[87.31; 91.94]	[89.97; 91.81]
	89.76	90.09	91.33	91.65
(2, 8)	[86.47; 91.34]	[89.28; 90.53]	[89.66; 92.56]	[90.41; 92.31]
	90.01	90.14	92.03	91.86
(2, 16)	[88.09; 90.75]	[89.00; 90.84]	[91.59; 92.53]	[90.84; 92.59]

plementations. A single-layer CMRU with $d = 16$ achieves 98.30% on wake-word detection, 95.38% on digit recognition, and 89.76% on full vocabulary classification.

H.5. Extended Parity Task Analysis

Table 8 presents complete results for the Parity task across all tested ε values for both CMRU and α CMRU variants. The results confirm that reflection dynamics ($\varepsilon = -1$) are necessary and sufficient for solving this task, while all other configurations fail to exceed chance performance. This finding directly supports the theoretical analysis connecting negative eigenvalues to state-tracking capabilities discussed in Section 6.

Table 8. Performance on the Parity task across different model types and ε values. Accuracy (%) on sequences of length $L \in [50, 1000]$ after training on $L \in [50, 400]$. Results shown as mean across five seeds with min–max range. **Top:** CMRU with $\varepsilon \in \{-1.00, 0.00, 0.10, 0.25, 0.50, 0.75, 1.00\}$ and minGRU. Note $\varepsilon = 0$ is the original BMRU. **Bottom:** α CMRU with the same ε values and LRU. Both CMRU and α CMRU achieve 100% with $\varepsilon = -1.00$; all other configurations perform at chance level (50%).

MODEL ε	CMRU -1.00	CMRU 0.00	CMRU 0.10	CMRU 0.25	CMRU 0.50	CMRU 0.75	CMRU 1.00	minGRU —
ACC. (%)	100.00 [100.00; 100.00]	50.01 [49.97; 50.06]	49.97 [49.34; 50.53]	49.89 [49.62; 50.09]	49.81 [49.53; 49.97]	49.95 [49.50; 50.47]	50.03 [49.47; 50.53]	49.95 [49.78; 50.03]
MODEL ε	α CMRU -1.00	α CMRU 0.00	α CMRU 0.10	α CMRU 0.25	α CMRU 0.50	α CMRU 0.75	α CMRU 1.00	LRU —
ACC. (%)	100.00 [100.00; 100.00]	49.82 [49.47; 50.03]	49.81 [49.31; 50.06]	50.10 [49.38; 50.81]	49.76 [49.12; 50.06]	49.81 [49.31; 50.00]	49.82 [49.28; 50.34]	49.88 [49.31; 50.53]

I. Tabulated Results for All Figures

This appendix reports results already presented elsewhere in the article in the form of figures. For the sake of scientific transparency, we provide below a summary of the quantitative results in table form, where each metric is reported by its average and the min–max interval observed over the five seeds tested. The raw data and codes are publicly available and contain more information, such as the evolution of metrics during training and the results on the validation datasets (those in the paper are on the held-out test dataset).

Table 9. Accuracy (%) for sMNIST with $d = 32$ across different epsilon values. Note that $\varepsilon = 0$ corresponds to the original BMRU. Figure 2, left panel.

MODEL TYPE	EPSILON	ACCURACY (%)
CMRU	0.00	30.13
		[26.94; 35.09]
CMRU	0.10	32.59
		[25.19; 39.84]
CMRU	0.25	41.54
		[37.69; 43.91]
CMRU	0.50	70.75
		[53.03; 81.81]
CMRU	0.75	89.40
		[83.47; 93.38]
CMRU	1.00	95.92
		[95.41; 96.19]
α CMRU	0.00	54.22
		[47.31; 59.50]
α CMRU	0.10	56.92
		[47.94; 70.72]
α CMRU	0.25	76.08
		[70.19; 85.09]
α CMRU	0.50	92.59
		[89.91; 93.66]
α CMRU	0.75	96.45
		[95.56; 96.91]
α CMRU	1.00	96.76
		[96.22; 97.00]

Table 10. Accuracy (%) for sMNIST with $\varepsilon = 1.0$ across different state dimensions. Figure 2, right panel.

MODEL TYPE	STATE DIM	EPSILON	ACCURACY (%)
α CMRU	4	1.00	88.34
			[87.38; 89.53]
α CMRU	8	1.00	93.49
			[91.56; 94.62]
α CMRU	16	1.00	95.43
			[93.03; 96.47]
α CMRU	32	1.00	96.76
			[96.22; 97.00]
α CMRU	64	1.00	96.88
			[96.66; 97.12]
α CMRU	128	1.00	97.16
			[96.72; 97.91]
α CMRU	256	1.00	97.25
			[96.69; 98.06]
CMRU	4	1.00	83.61
			[80.97; 85.09]
CMRU	8	1.00	91.49
			[90.53; 92.69]
CMRU	16	1.00	94.83
			[93.12; 95.84]
CMRU	32	1.00	95.92
			[95.41; 96.19]
CMRU	64	1.00	97.05
			[96.69; 97.28]
CMRU	128	1.00	96.20
			[95.16; 97.00]
CMRU	256	1.00	96.16
			[95.81; 96.38]
LRU	4	—	72.88
			[53.87; 83.22]
LRU	8	—	78.08
			[62.72; 88.34]
LRU	16	—	88.98
			[82.78; 91.53]
LRU	32	—	91.97
			[89.84; 93.78]
LRU	64	—	92.59
			[90.41; 93.84]
LRU	128	—	94.08
			[91.66; 96.06]
LRU	256	—	95.61
			[94.69; 96.31]
MINGRU	4	—	75.88
			[66.16; 86.09]
MINGRU	8	—	81.88
			[73.28; 88.69]
MINGRU	16	—	92.17
			[90.34; 93.28]
MINGRU	32	—	92.01
			[89.50; 96.16]
MINGRU	64	—	93.39
			[88.50; 96.47]
MINGRU	128	—	93.88
			[91.12; 96.00]
MINGRU	256	—	96.62
			[94.62; 97.62]

Table 11. MAE for Copy First Continuous (noisy) tasks across architectures (d , r) and sequence lengths (Part 1 of 3). Figure 3.

MODEL TYPE	SEQ LENGTH	STATE DIM	NUM RECS	MAE
α CMRU	100	4	1	0.0002 [0.0002; 0.0002]
α CMRU	300	4	1	0.0002 [0.0001; 0.0002]
α CMRU	500	4	1	0.0002 [0.0002; 0.0002]
α CMRU	1000	4	1	0.0002 [0.0002; 0.0003]
α CMRU	2000	4	1	0.0002 [0.0002; 0.0002]
α CMRU	5000	4	1	0.0005 [0.0002; 0.0014]
α CMRU	10000	4	1	0.1009 [0.0002; 0.5038]
CMRU	100	4	1	0.0401 [0.0309; 0.0466]
CMRU	300	4	1	0.0404 [0.0342; 0.0457]
CMRU	500	4	1	0.0383 [0.0362; 0.0399]
CMRU	1000	4	1	0.0402 [0.0362; 0.0432]
CMRU	2000	4	1	0.1743 [0.0378; 0.5037]
CMRU	5000	4	1	0.1538 [0.0363; 0.5038]
CMRU	10000	4	1	0.1357 [0.0352; 0.5037]
LRU	100	4	1	0.1011 [0.0002; 0.5038]
LRU	300	4	1	0.3025 [0.0005; 0.5038]
LRU	500	4	1	0.4031 [0.0004; 0.5040]
LRU	1000	4	1	0.4033 [0.0012; 0.5039]
LRU	2000	4	1	0.5039 [0.5038; 0.5044]
LRU	5000	4	1	0.5039 [0.5037; 0.5042]
LRU	10000	4	1	0.5038 [0.5037; 0.5038]
MINGRU	100	4	1	0.4031 [0.0002; 0.5039]
MINGRU	300	4	1	0.5038 [0.5038; 0.5039]
MINGRU	500	4	1	0.5038 [0.5038; 0.5038]
MINGRU	1000	4	1	0.5038 [0.5038; 0.5039]
MINGRU	2000	4	1	0.5039 [0.5038; 0.5042]
MINGRU	5000	4	1	0.5038 [0.5037; 0.5041]
MINGRU	10000	4	1	0.5037 [0.5036; 0.5038]
α CMRU	100	16	1	0.0002 [0.0002; 0.0002]
α CMRU	300	16	1	0.0002 [0.0002; 0.0002]
α CMRU	500	16	1	0.0002 [0.0002; 0.0002]
α CMRU	1000	16	1	0.0002 [0.0002; 0.0002]

Table 12. MAE for Copy First Continuous (noisy) tasks across architectures (d , r) and sequence lengths (Part 2 of 3). Figure 3.

MODEL TYPE	SEQ LENGTH	STATE DIM	NUM RECS	MAE
α CMRU	2000	16	1	0.0002 [0.0001; 0.0002]
α CMRU	5000	16	1	0.0002 [0.0002; 0.0003]
α CMRU	10000	16	1	0.1010 [0.0002; 0.5037]
CMRU	100	16	1	0.0119 [0.0099; 0.0143]
CMRU	300	16	1	0.0143 [0.0112; 0.0163]
CMRU	500	16	1	0.0147 [0.0139; 0.0155]
CMRU	1000	16	1	0.0148 [0.0128; 0.0159]
CMRU	2000	16	1	0.0154 [0.0122; 0.0186]
CMRU	5000	16	1	0.1142 [0.0168; 0.5038]
CMRU	10000	16	1	0.1142 [0.0168; 0.5038]
LRU	100	16	1	0.0002 [0.0002; 0.0003]
LRU	300	16	1	0.0004 [0.0002; 0.0006]
LRU	500	16	1	0.1012 [0.0003; 0.5038]
LRU	1000	16	1	0.2021 [0.0003; 0.5039]
LRU	2000	16	1	0.5039 [0.5038; 0.5041]
LRU	5000	16	1	0.5039 [0.5037; 0.5046]
LRU	10000	16	1	0.5037 [0.5036; 0.5038]
MINGRU	100	16	1	0.3023 [0.0002; 0.5038]
MINGRU	300	16	1	0.4030 [0.0002; 0.5038]
MINGRU	500	16	1	0.5038 [0.5038; 0.5038]
MINGRU	1000	16	1	0.5038 [0.5037; 0.5039]
MINGRU	2000	16	1	0.5039 [0.5038; 0.5042]
MINGRU	5000	16	1	0.5039 [0.5038; 0.5043]
MINGRU	10000	16	1	0.5038 [0.5037; 0.5038]
α CMRU	100	16	4	0.0001 [0.0001; 0.0002]
α CMRU	300	16	4	0.0001 [0.0001; 0.0002]
α CMRU	500	16	4	0.0001 [0.0001; 0.0002]
α CMRU	1000	16	4	0.0001 [0.0001; 0.0002]
α CMRU	2000	16	4	0.0001 [0.0000; 0.0001]
α CMRU	5000	16	4	0.0001 [0.0001; 0.0002]
α CMRU	10000	16	4	0.0004 [0.0001; 0.0014]
CMRU	100	16	4	0.0019 [0.0014; 0.0030]

Table 13. MAE for Copy First Continuous (noisy) tasks across architectures (d, r) and sequence lengths (Part 3 of 3). Figure 3.

MODEL TYPE	SEQ LENGTH	STATE DIM	NUM RECS	MAE
CMRU	300	16	4	0.0036 [0.0021; 0.0048]
CMRU	500	16	4	0.0034 [0.0020; 0.0050]
CMRU	1000	16	4	0.0032 [0.0023; 0.0049]
CMRU	2000	16	4	0.0034 [0.0026; 0.0040]
CMRU	5000	16	4	0.0049 [0.0027; 0.0128]
CMRU	10000	16	4	0.0066 [0.0033; 0.0134]
LRU	100	16	4	0.0002 [0.0002; 0.0003]
LRU	300	16	4	0.0003 [0.0001; 0.0005]
LRU	500	16	4	0.0006 [0.0002; 0.0011]
LRU	1000	16	4	0.1022 [0.0003; 0.5038]
LRU	2000	16	4	0.4038 [0.0038; 0.5038]
LRU	5000	16	4	0.5038 [0.5038; 0.5040]
LRU	10000	16	4	0.5038 [0.5037; 0.5038]
MINGRU	100	16	4	0.1012 [0.0001; 0.5038]
MINGRU	300	16	4	0.1013 [0.0003; 0.5038]
MINGRU	500	16	4	0.2021 [0.0002; 0.5038]
MINGRU	1000	16	4	0.5038 [0.5038; 0.5038]
MINGRU	2000	16	4	0.5039 [0.5038; 0.5046]
MINGRU	5000	16	4	0.5039 [0.5038; 0.5044]
MINGRU	10000	16	4	0.5039 [0.5038; 0.5043]

Table 14. Accuracy (%) comparison across sequential MNIST permutations for $d = 32$ with different epsilon values (Part 1 of 2). Note that $\varepsilon = 0$ corresponds to the original BMRU. Figure 6.

TASK NAME	MODEL TYPE	EPSILON	ACCURACY (%)
SMNIST	CMRU	0.00	30.13 [26.94; 35.09]
SMNIST	CMRU	0.10	32.59 [25.19; 39.84]
SMNIST	CMRU	0.25	41.54 [37.69; 43.91]
SMNIST	CMRU	0.50	70.75 [53.03; 81.81]
SMNIST	CMRU	0.75	89.40 [83.47; 93.38]
SMNIST	CMRU	1.00	95.92 [95.41; 96.19]
SMNIST	α CMRU	0.00	54.22 [47.31; 59.50]
SMNIST	α CMRU	0.10	56.92 [47.94; 70.72]
SMNIST	α CMRU	0.25	76.08 [70.19; 85.09]
SMNIST	α CMRU	0.50	92.59 [89.91; 93.66]
SMNIST	α CMRU	0.75	96.45 [95.56; 96.91]
SMNIST	α CMRU	1.00	96.76 [96.22; 97.00]
SMNIST	LRU	—	91.97 [89.84; 93.78]
SMNIST	MINGRU	—	92.01 [89.50; 96.16]
PMNIST	CMRU	0.00	52.98 [48.81; 55.31]
PMNIST	CMRU	0.10	67.49 [64.91; 70.78]
PMNIST	CMRU	0.25	77.87 [72.12; 83.22]
PMNIST	CMRU	0.50	84.62 [77.12; 91.62]
PMNIST	CMRU	0.75	91.32 [89.81; 93.31]
PMNIST	CMRU	1.00	95.06 [94.56; 95.94]
PMNIST	α CMRU	0.00	83.58 [82.28; 84.59]
PMNIST	α CMRU	0.10	85.56 [81.38; 89.44]
PMNIST	α CMRU	0.25	91.19 [89.78; 92.22]
PMNIST	α CMRU	0.50	93.99 [93.03; 94.69]
PMNIST	α CMRU	0.75	95.99 [95.38; 96.53]
PMNIST	α CMRU	1.00	95.39 [92.97; 96.38]
PMNIST	LRU	—	96.12 [95.75; 96.47]
PMNIST	MINGRU	—	95.35 [94.03; 96.00]
PMNIST ₂₄	CMRU	0.00	55.31 [49.38; 61.62]
PMNIST ₂₄	CMRU	0.10	67.64 [61.31; 72.72]
PMNIST ₂₄	CMRU	0.25	70.71 [65.78; 73.44]
PMNIST ₂₄	CMRU	0.50	85.63 [77.62; 89.25]

Table 15. Accuracy (%) comparison across sequential MNIST permutations for $d = 32$ with different epsilon values (Part 2 of 2). Note that $\varepsilon = 0$ corresponds to the original BMRU. Figure 6.

TASK NAME	MODEL TYPE	EPSILON	ACCURACY (%)
PMNIST ₂₄	CMRU	0.75	91.09 [90.50; 91.81]
PMNIST ₂₄	CMRU	1.00	93.52 [92.38; 95.12]
PMNIST ₂₄	α CMRU	0.00	82.78 [75.03; 86.41]
PMNIST ₂₄	α CMRU	0.10	86.70 [83.69; 89.84]
PMNIST ₂₄	α CMRU	0.25	89.03 [87.91; 89.81]
PMNIST ₂₄	α CMRU	0.50	94.42 [92.94; 95.59]
PMNIST ₂₄	α CMRU	0.75	95.80 [94.16; 96.91]
PMNIST ₂₄	α CMRU	1.00	95.77 [93.56; 96.69]
PMNIST ₂₄	LRU	—	95.74 [95.34; 96.25]
PMNIST ₂₄	MINGRU	—	95.69 [95.19; 96.25]
PMNIST ₃₆	CMRU	0.00	47.07 [38.62; 59.66]
PMNIST ₃₆	CMRU	0.10	59.73 [56.19; 63.72]
PMNIST ₃₆	CMRU	0.25	73.03 [68.97; 80.25]
PMNIST ₃₆	CMRU	0.50	88.22 [82.62; 91.44]
PMNIST ₃₆	CMRU	0.75	92.09 [89.66; 93.66]
PMNIST ₃₆	CMRU	1.00	92.81 [90.62; 93.81]
PMNIST ₃₆	α CMRU	0.00	82.20 [79.66; 84.44]
PMNIST ₃₆	α CMRU	0.10	87.90 [85.81; 89.62]
PMNIST ₃₆	α CMRU	0.25	92.01 [88.25; 94.53]
PMNIST ₃₆	α CMRU	0.50	95.20 [94.47; 95.97]
PMNIST ₃₆	α CMRU	0.75	96.46 [96.22; 96.84]
PMNIST ₃₆	α CMRU	1.00	96.06 [95.56; 97.00]
PMNIST ₃₆	LRU	—	95.86 [95.03; 96.75]
PMNIST ₃₆	MINGRU	—	95.64 [94.94; 96.56]

Table 16. Accuracy (%) for Copy First Discrete (15 items) tasks across sequence lengths. Figure 7, left panel.

MODEL TYPE	SEQ LENGTH	ACCURACY (%)
CMRU	100	100.00 [100.00; 100.00]
α CMRU	100	100.00 [100.00; 100.00]
LRU	100	100.00 [100.00; 100.00]
MINGRU	100	25.48 [6.28; 100.00]
CMRU	300	100.00 [100.00; 100.00]
α CMRU	300	100.00 [100.00; 100.00]
LRU	300	62.61 [6.41; 100.00]
MINGRU	300	6.67 [6.28; 7.34]
CMRU	500	100.00 [100.00; 100.00]
α CMRU	500	100.00 [100.00; 100.00]
LRU	500	44.04 [6.50; 100.00]
MINGRU	500	6.71 [6.50; 7.34]
CMRU	1000	100.00 [100.00; 100.00]
α CMRU	1000	100.00 [100.00; 100.00]
LRU	1000	25.31 [6.28; 100.00]
MINGRU	1000	6.49 [6.28; 6.66]
CMRU	2000	100.00 [100.00; 100.00]
α CMRU	2000	100.00 [100.00; 100.00]
LRU	2000	6.53 [6.12; 7.34]
MINGRU	2000	6.99 [6.50; 7.34]
CMRU	5000	98.74 [93.72; 100.00]
α CMRU	5000	100.00 [100.00; 100.00]
LRU	5000	6.81 [6.41; 7.34]
MINGRU	5000	6.50 [6.28; 6.66]
CMRU	10000	100.00 [100.00; 100.00]
α CMRU	10000	100.00 [100.00; 100.00]
LRU	10000	6.50 [6.28; 6.66]
MINGRU	10000	6.64 [6.28; 7.34]

Table 17. MAE for Copy First Continuous (no noise) tasks with $d = 4$ across sequence lengths. Figure 7, right panel.

MODEL TYPE	SEQ LENGTH	MAE
CMRU	100	0.0424 [0.0392; 0.0465]
α CMRU	100	0.0002 [0.0002; 0.0002]
LRU	100	0.0002 [0.0002; 0.0003]
MINGRU	100	0.3023 [0.0002; 0.5038]
CMRU	300	0.0412 [0.0380; 0.0438]
α CMRU	300	0.0002 [0.0001; 0.0002]
LRU	300	0.2017 [0.0001; 0.5041]
MINGRU	300	0.4031 [0.0002; 0.5038]
CMRU	500	0.0443 [0.0322; 0.0539]
α CMRU	500	0.0002 [0.0002; 0.0002]
LRU	500	0.2016 [0.0002; 0.5038]
MINGRU	500	0.5038 [0.5038; 0.5039]
CMRU	1000	0.0465 [0.0397; 0.0525]
α CMRU	1000	0.0002 [0.0002; 0.0003]
LRU	1000	0.4031 [0.0002; 0.5039]
MINGRU	1000	0.5038 [0.5038; 0.5039]
CMRU	2000	0.0432 [0.0361; 0.0499]
α CMRU	2000	0.0002 [0.0002; 0.0002]
LRU	2000	0.4032 [0.0006; 0.5038]
MINGRU	2000	0.5038 [0.5038; 0.5038]
CMRU	5000	0.0470 [0.0355; 0.0618]
α CMRU	5000	0.0002 [0.0002; 0.0002]
LRU	5000	0.5038 [0.5038; 0.5039]
MINGRU	5000	0.5038 [0.5038; 0.5038]
CMRU	10000	0.0589 [0.0538; 0.0698]
α CMRU	10000	0.0002 [0.0002; 0.0002]
LRU	10000	0.5038 [0.5038; 0.5039]
MINGRU	10000	0.5038 [0.5038; 0.5038]